

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI THÀNH PHỐ HỒ CHÍ MINH
VIỆN CÔNG NGHỆ THÔNG TIN VÀ ĐIỆN, ĐIỆN TỬ



BÁO CÁO MÔN HỌC
NGÔN NGỮ LẬP TRÌNH PYTHON

TÊN ĐỀ TÀI
HỆ THỐNG QUẢN LÝ CHI TIÊU: PHÂN TÍCH VÀ DỰ
BÁO HÀNH VI TÀI CHÍNH CÁ NHÂN

Thành phố Hồ Chí Minh, tháng 6 năm 2026

MỤC LỤC

LỜI MỞ ĐẦU.....	1
LỜI CẢM ƠN.....	2
CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI.....	3
1.1. LÝ DO CHỌN ĐỀ TÀI.....	3
1.2. MỤC TIÊU	3
1.3. TỔNG QUAN VỀ DỮ LIỆU	3
1.3.1. Nguồn dữ liệu.....	3
1.3.2. Mô tả dữ liệu.....	4
1.4. KỸ THUẬT SỬ DỤNG.....	9
CHƯƠNG 2: MÔ PHỎNG, KHÁM PHÁ VÀ TIỀN XỬ LÝ DỮ LIỆU ĐA NGUỒN.....	10
2.1. KHỞI TẠO VÀ NẠP DỮ LIỆU GỐC.....	10
2.2. QUY TRÌNH MÔ PHỎNG DỮ LIỆU THÔ.....	10
2.2.1. Hệ thống giao dịch nhỏ (Income - Expenses - Budget)	10
2.2.2. Tập hành vi mở rộng (8000_extended)	11
2.2.3. Tập chỉ số vĩ mô máy học (Tracker_dataset)	11
2.3. PHÂN TÍCH KHÁM PHÁ (EDA) & XÁC ĐỊNH LỖ HỔNG DỮ LIỆU.....	12
2.3.1. Khởi tạo và nạp nguồn dữ liệu mô phỏng nhiều	12
2.3.2. Kiểm toán quy mô và định dạng sơ đồ dữ liệu	13
2.3.3. Dò tìm và định lượng khoảng trống dữ liệu.....	16
2.3.4. Rà soát tần suất trùng lặp và đo lường thống kê.....	18
2.3.5. Khai phá và cô lập các lỗ hổng hệ thống.....	20
2.4. TIỀN XỬ LÝ VÀ LÀM SẠCH DỮ LIỆU	22
2.4.1. Chuẩn hóa và làm sạch khối dữ liệu giao dịch lẻ (Expenses - Income - Budget)	22
2.4.2. Xử lý giá trị phi lý và khử nhiễu ngoại lai bằng IQR trên tập Tracker.....	24
2.4.3. Xử lý nhiễu diện rộng và đồng bộ từ vựng trên Tập hành vi mở rộng 8000	31
2.5. ĐỒNG BỘ VÀ LƯU TRỮ KHO DỮ LIỆU SẠCH VÀO MYSQL.....	35
CHƯƠNG 3: PHÂN TÍCH KHÁM PHÁ DỮ LIỆU TRỰC QUAN	39
3.1. CÁN CÂN THANH KHOẢN TIỀN MẶT NGẮN HẠN.....	39
3.2. BIÊN ĐỘ LỆCH RÒNG GIỮA THỰC CHI VỚI HẠN MỨC NGÂN SÁCH	41
3.3. DANH MỤC LŨY KẾ TIÊU TỐN TIỀN MẶT DÀI HẠN	43
3.4. BIÊN ĐỘ CHI TIÊU THEO BIẾN ĐỊNH TÍNH ÁP LỰC VÀ NỢ	45
3.5. MA TRẬN TƯƠNG QUAN TUYẾN TÍNH ĐA BIẾN	47

3.6. MẬT ĐỘ PHÂN PHỐI ĐIỂM TÍN NHIỆM TRONG QUẦN THỂ.....	49
3.7. TƯƠNG QUAN THU NHẬP VỚI MỨC ĐỘ VUNG TAY CHI TIÊU.....	51
3.8. ĐỀ XUẤT GIẢI PHÁP.....	52
CHƯƠNG 4: PHÂN TÍCH DỮ LIỆU ĐƯA VÀO MÔ HÌNH MÁY HỌC	54
4.1. TIỀN XỬ LÝ VÀ MÃ HÓA	54
4.1.1. Đọc dữ liệu	55
4.1.2. Loại bỏ dữ liệu không đầy đủ.....	55
4.1.3. Lựa chọn đặc trưng đầu vào	56
4.1.4. Kiểm tra sự tồn tại của các cột.....	57
4.1.5. Xử lý giá trị khuyết thiếu.....	57
4.1.6. Xây dựng ma trận đặc trưng.....	58
4.1.7. Xây dựng biến mục tiêu	59
4.2. MÃ HÓA DỮ LIỆU VÀ XÂY DỰNG BIẾN MỤC TIÊU	60
4.2.1. Mã hóa biến mục tiêu cho bài toán phân loại	60
4.2.2. Lưu tên các lớp.....	61
4.2.3. Xây dựng biến mục tiêu cho bài toán hồi quy	62
4.3. CHIA TẬP DỮ LIỆU VÀ CHUẨN HÓA DỮ LIỆU	62
4.3.1. Chia dữ liệu Train và Test.....	63
4.3.2. Chuẩn hóa dữ liệu	64
4.4. XÂY DỰNG VÀ HUẤN LUYỆN CÁC MÔ HÌNH HỌC MÁY	66
4.4.1. Khởi tạo các mô hình	66
4.4.2. Huấn luyện và đánh giá.....	67
CHƯƠNG 5: ĐÁNH GIÁ VÀ NHẬN XÉT MÔ HÌNH HỌC MÁY	69
5.1. ĐÁNH GIÁ MÔ HÌNH LOGISTIC REGRESSION	69
5.1.1. Kết quả mô hình Logistic Regression	69
5.1.2. Nhận xét.....	69
5.2. ĐÁNH GIÁ MÔ HÌNH DECISION TREE	70
5.2.1. Kết quả mô hình Decision Tree	70
5.2.2. Nhận xét.....	71
5.3. ĐÁNH GIÁ MÔ HÌNH BALANCED RANDOM FOREST	72
5.3.1. Kết quả mô hình Balanced Random Forest.....	72
5.3.2. Nhận xét.....	72
5.4. ĐÁNH GIÁ MÔ HÌNH HỒI QUY	73

5.5. SO SÁNH CÁC MÔ HÌNH	75
5.5.1. So sánh các mô hình phân loại.....	75
5.5.2. Nhận xét.....	75
CHƯƠNG 6: TỔNG KẾT	77
TÀI LIỆU THAM KHẢO	78

LỜI MỞ ĐẦU

Trong thời đại số hóa hiện nay, dữ liệu tài chính cá nhân ngày càng được tạo ra với khối lượng lớn từ nhiều hoạt động khác nhau như thu nhập, chi tiêu, tiết kiệm và lập ngân sách. Tuy nhiên, các dữ liệu này thường được ghi nhận từ nhiều nguồn khác nhau và tồn tại dưới nhiều định dạng khác nhau, dẫn đến khó khăn trong quá trình tổng hợp, phân tích và đánh giá tình hình tài chính.

Bên cạnh đó, dữ liệu tài chính trong thực tế thường xuất hiện nhiều vấn đề như dữ liệu thiếu, dữ liệu trùng lặp, lỗi nhập liệu và các giá trị bất thường. Những lỗi này có thể ảnh hưởng đáng kể đến độ chính xác của kết quả phân tích nếu không được phát hiện và xử lý kịp thời.

Xuất phát từ thực tế đó, nhóm thực hiện đề tài nhằm xây dựng một quy trình xử lý dữ liệu tài chính cá nhân dựa trên ba nguồn dữ liệu chính gồm dữ liệu thu nhập (Income), dữ liệu chi tiêu (Expenses) và dữ liệu ngân sách (Budget). Hệ thống thực hiện việc tích hợp dữ liệu, khám phá cấu trúc dữ liệu thô, phát hiện các lỗi tồn tại trong dữ liệu và chuẩn bị dữ liệu cho các bước phân tích và xây dựng mô hình học máy ở các giai đoạn tiếp theo.

Dữ liệu sau khi được xử lý sẽ giúp phản ánh chính xác hơn tình hình tài chính cá nhân, đồng thời tạo nền tảng cho việc trực quan hóa dữ liệu và dự đoán mức độ áp lực tài chính bằng các thuật toán học máy.

LỜI CẢM ƠN

Dù đã rất cố gắng nhưng bài báo cáo của nhóm sẽ không tránh khỏi thiếu sót. Nhóm chúng em rất mong nhận được những góp ý của Thầy để đề tài hoàn thiện hơn.

Chúng em xin chân thành cảm ơn!

CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI

1.1. LÝ DO CHỌN ĐỀ TÀI

Việc lựa chọn đề tài được thực hiện dựa trên những lý do sau:

- Thứ nhất, quản lý tài chính cá nhân đang trở thành một nhu cầu thiết yếu trong cuộc sống hiện đại. Việc theo dõi thu nhập, chi tiêu và ngân sách giúp mỗi cá nhân kiểm soát tài chính hiệu quả hơn và đưa ra các quyết định chi tiêu hợp lý.
- Thứ hai, dữ liệu tài chính thường chứa nhiều lỗi phát sinh trong quá trình thu thập và nhập liệu. Các lỗi này bao gồm dữ liệu thiếu, dữ liệu trùng lặp, lỗi chính tả hoặc các giá trị bất thường. Nếu không được xử lý đúng cách, các lỗi này sẽ làm giảm độ tin cậy của kết quả phân tích.
- Thứ ba, đề tài tạo điều kiện để áp dụng các kiến thức về khoa học dữ liệu như tiền xử lý dữ liệu, khám phá dữ liệu, trực quan hóa dữ liệu và học máy vào một bài toán thực tế.
- Thứ tư, việc xây dựng quy trình xử lý dữ liệu hoàn chỉnh giúp tạo ra nguồn dữ liệu chất lượng cao, phục vụ cho các bước phân tích tài chính và dự đoán trong tương lai.

1.2. MỤC TIÊU

Xây dựng hệ thống tự động hóa quy trình xử lý dữ liệu tài chính đa nguồn từ khâu làm sạch, lưu trữ tập trung đến khai phá trực quan hành vi chi tiêu. Đồng thời, ứng dụng và kiểm thử mô hình học máy để dự báo khả năng hoàn thành kế hoạch tích lũy của người dùng, đảm bảo hệ thống vận hành ổn định trong môi trường thực tế.

1.3. TỔNG QUAN VỀ DỮ LIỆU

1.3.1. Nguồn dữ liệu

[Personal Finance Tracker Dataset](#)

[Personal Budget Transactions Dataset](#)

Personal Finance Dataset

Financial Transactions Dataset (Expenses & Income)

1.3.2. Mô tả dữ liệu

1.3.2.1. Financial Transactions Dataset (Expenses & Income)

Hai bảng này đồng bộ cấu trúc với nhau đều ghi nhận chi tiết từng phát sinh giao dịch.

Thuộc tính	Mô tả	Kiểu dữ liệu
date_time	Ngày và thời gian phát sinh giao dịch thu nhập/chi tiêu.	Datetime
category	Danh mục phân loại	String/Text
account	Tài khoản thực hiện giao dịch	String
amount	Số tiền của giao dịch phát sinh.	Float/Numeric
currency	Đơn vị tiền tệ sử dụng	String
tags	Thẻ phân loại mở rộng phục vụ tìm kiếm nhanh.	String

1.3.2.2. Personal Budget Transactions Dataset

Bảng này ghi nhận kế hoạch hạn mức dòng tiền được thiết lập theo danh mục

Thuộc tính	Mô tả	Kiểu dữ liệu
date	Ngày thiết lập hạn mức hoặc ngày ghi nhận ngân sách.	Datetime
category	Danh mục áp dụng hạn mức ngân sách	String/Text
amount	Số tiền ngân sách được phân bổ tối đa cho danh mục đó.	Float/Numeric

1.3.2.3. *Personal Finance Tracker Dataset*

Bảng dữ liệu tích hợp sâu chứa các chỉ số tài chính vĩ mô của người dùng

Thuộc tính	Mô Tả	Kiểu dữ liệu
date	Ngày ghi nhận các chỉ số tài chính trong tháng.	Datetime
user_id	Mã định danh duy nhất của người dùng hệ thống.	Integer
monthly_income	Tổng thu nhập bình quân trong tháng.	Float
monthly_expense_total	Tổng số tiền đã chi tiêu thực tế trong tháng.	Float
savings_rate	Tỷ lệ tiết kiệm thực tế (Số tiền tiết kiệm / Thu nhập).	Float

budget_goal	Mục tiêu ngân sách tiết kiệm đề ra đầu tháng.	Float
financial_scenario	Kịch bản kinh tế tác động	String
credit_score	Điểm tín dụng cá nhân (Đánh giá mức độ uy tín tài chính).	Float
debt_to_income_ratio	Tỷ lệ nợ trên thu nhập (Hệ số DTI).	Float
loan_payment	Số tiền phải trả cho các khoản vay/trả góp trong tháng.	Float
investment_amount	Số tiền phân bổ vào các kênh đầu tư.	Float
subscription_services	Số lượng dịch vụ đăng ký trả phí định kỳ	Integer
emergency_fund	Giá trị hiện tại của quỹ dự phòng khẩn cấp.	Float
transaction_count	Tổng số lượng giao dịch phát sinh trong tháng.	Integer
fraud_flag	Cảnh báo phát hiện giao dịch gian lận hoặc bất thường.	Binary (0/1)

discretionary_spen	Chi tiêu linh hoạt (các khoản chi không bắt buộc: giải trí, mua sắm).	Float
essential_spending	Chi tiêu thiết yếu (các khoản bắt buộc: tiền nhà, ăn uống, hóa đơn).	Float
income_type	Nguồn hình thành thu nhập	String
rent_or_mortgage	Chi phí thuê nhà hoặc trả gốc vay mua nhà hàng tháng.	Float
category	Danh mục chi tiêu chiếm tỷ trọng lớn nhất trong tháng.	String
cash_flow_status	Trạng thái dòng tiền	String
financial_advice_score	Điểm số đánh giá dựa trên lời khuyên từ chuyên gia tài chính.	Float
financial_stress_level	Mức độ áp lực tài chính của cá nhân (Low, Medium, High).	String
actual_savings	Số tiền tích lũy thực tế giữ lại được cuối tháng.	Float

savings_goal_met	Trạng thái đạt mục tiêu tiết kiệm (1: Có, 0: Không).	Binary (0/1)
------------------	--	--------------

1.3.2.4. Personal Finance Dataset

Bảng dữ liệu đặc thù ghi nhận chi tiết 8.000 hành vi giao dịch, thanh toán và tiêu dùng mở rộng của người dùng.

Thuộc tính	Mô tả	Kiểu dữ liệu
Date	Ngày phát sinh giao dịch tài chính.	Datetime
Description	Mô tả chi tiết nội dung giao dịch	String/Text
Amount	Số tiền phát sinh của giao dịch.	Float/Numeric
Category	Danh mục chi tiêu chính	String
PaymentMethod	Phương thức thanh toán	String
Location	Địa điểm hoặc thành phố phát sinh giao dịch	String
AccountType	Loại tài khoản sử dụng	String
TransactionType	Loại giao dịch	String

DeviceUsed	Thiết bị thực hiện giao dịch	String
Currency	Đơn vị tiền tệ	String
MerchantType	Loại hình đại lý/thương mại	String
LoyaltyProgram	Giao dịch có áp dụng chương trình khách hàng thân thiết hay không.	String (Yes/No)
Weekday	Ngày trong tuần phát sinh giao dịch	String
Month	Tháng phát sinh giao dịch trong năm	String
TimeOfDay	Khung thời gian trong ngày	String

1.4. KỸ THUẬT SỬ DỤNG

Công cụ khai thác dữ liệu: VS Code

Phiên bản Python: 3.14.5.

Thư viện: Numpy, Pandas, Seaborn, Sklearn, Matplotlib, Pathlib

Mô hình sử dụng: RandomForest, Logistic, DecisionTree

CHƯƠNG 2: MÔ PHỎNG, KHÁM PHÁ VÀ TIỀN XỬ LÝ DỮ LIỆU ĐA NGUỒN

2.1. KHỞI TẠO VÀ NẠP DỮ LIỆU GỐC

```
import pandas as pd
import numpy as np
import random
np.random.seed(42)
random.seed(42)
try:
    df_expenses_raw = pd.read_csv('Expenses_clean.csv')
    df_income_raw = pd.read_csv('Income_clean.csv')
    df_budget_raw = pd.read_csv('budget_data.csv')
    df_tracker_raw = pd.read_csv('personal_finance_tracker_dataset.csv')
    df_8000_raw = pd.read_csv('personal_finance_dataset_8000_extended.csv')
except FileNotFoundError as e:
    print(f" Thiếu file trên hệ thống. Vui lòng cập nhật file vào! {e}")
    raise
```

Hệ thống tiến hành cấu hình môi trường bằng cách khai báo các thư viện lỗi (pandas, numpy) và thiết lập cơ chế đọc đồng thời 5 tệp dữ liệu thô (.csv) đại diện cho các khía cạnh: Lịch sử thu nhập cá nhân (Income), Nhật ký chi tiêu giao dịch (Expenses), Hạn mức kế hoạch tuần/tháng (Budget), Hồ sơ chỉ số tài chính vĩ mô (Tracker) và Tập hành vi tiêu dùng mở rộng 8.000 bản ghi.

2.2. QUY TRÌNH MÔ PHỎNG DỮ LIỆU THÔ

Nhằm tạo ra một môi trường thực nghiệm khách quan và giả lập chính xác các lỗi hệ sinh thái dữ liệu thực tế, hệ thống tiến hành can thiệp có ý, phá vỡ cấu trúc sạch của 5 tệp dữ liệu gốc thông qua 3 phân lớp nhiễu.

2.2.1. Hệ thống giao dịch nhỏ (*Income - Expenses - Budget*)

```
#1. Gây nhiễu file Expenses - Income - Budget
# Tạo ô trống ngẫu nhiên ở cột số tiền - Amount và danh mục - Category
mask_exp_amt = np.random.rand(len(df_expenses_raw)) < 0.05
mask_exp_cat = np.random.rand(len(df_expenses_raw)) < 0.04
mask_inc_amt = np.random.rand(len(df_income_raw)) < 0.05
mask_inc_cat = np.random.rand(len(df_income_raw)) < 0.05
df_budget_raw.loc[np.random.rand(len(df_budget_raw)) < 0.03, 'amount'] = np.nan
df_expenses_raw.loc[mask_exp_amt, 'amount'] = np.nan
df_expenses_raw.loc[mask_exp_cat, 'category'] = np.nan
df_income_raw.loc[mask_inc_amt, 'amount'] = np.nan
df_income_raw.loc[mask_inc_cat, 'category'] = np.nan
# Gây lỗi định dạng ngày tháng
df_expenses_raw['date_time'] = (df_expenses_raw['date_time'].astype(str).apply(lambda x: "ERR_DATE_FORMAT_99" if random.random() < 0.03 else x)
)
df_income_raw['date_time'] = df_income_raw['date_time'].astype(str).apply(lambda x: "ERR_DATE_FORMAT_99" if random.random() < 0.04 else x
)
#Lỗi Trùng lặp
ty_le_trung_exp = 0.10
so_dong_trung_exp = int(len(df_expenses_raw) * ty_le_trung_exp)
df_dup_exp = df_expenses_raw.sample(n = so_dong_trung_exp, random_state = 12)
df_exp_noisy = pd.concat([df_expenses_raw, df_dup_exp], ignore_index = True)
df_exp_noisy = df_exp_noisy.sample(frac = 1, random_state = 12).reset_index(drop = True)
```

Thêm các ô trống khuyết thiếu (NaN) ngẫu nhiên từ 3% đến 5% vào trường số tiền và danh mục; ghi đè chuỗi lỗi cấu trúc "ERR_DATE_FORMAT_99" vào trường mốc thời gian; đồng thời nhân bản dữ liệu mẫu để tạo 10% tỷ lệ bản ghi trùng lặp (Duplicate Noise).

2.2.2. Tập hành vi mở rộng (8000_extended)

```
#2. Gây nhiễu cho file 8000_extended
# Tạo giá trị trống khuyết thiếu ngẫu nhiên ở cột số tiền - Amount và danh mục - Category
mask_amt = np.random.rand(len(df_8000_raw)) < 0.05
mask_cat = np.random.rand(len(df_8000_raw)) < 0.04
df_8000_raw['Amount'] = df_8000_raw['Amount'].mask(mask_amt)
df_8000_raw['Category'] = df_8000_raw['Category'].mask(mask_cat)
# Tạo thêm giao dịch bất thường với dòng tiền lớn hơn thực tế
outlier_idx = np.random.choice(df_8000_raw.index, size = 50, replace = False)
df_8000_raw.loc[outlier_idx, 'Amount'] = df_8000_raw.loc[outlier_idx, 'Amount'] * 100
# Tạo lỗi ký tự và chính tả ở danh mục
df_8000_raw['Category'] = df_8000_raw['Category'].replace({'Travel': 'Travell_err', 'Grocery': 'Grocerie$', 'Bills': 'Biills'})
```

Áp dụng mặt nạ mask() để xóa dữ liệu ngẫu nhiên; nhân hệ số 100 lần trên 50 bản ghi được chọn ngẫu nhiên để tạo các điểm dị biệt dòng tiền (Outliers Noise); đồng thời cố ý sửa đổi danh mục thành các từ sai ký tự đặc biệt và chính tả ('Travell_err', 'Grocerie\$', 'Biills').

2.2.3. Tập chỉ số vĩ mô máy học (Tracker_dataset)

```
#3. Gây nhiễu cho file Tracker_dataset
# Tạo giá trị trống hệ thống ở các đặc trưng cốt lõi
for col in ['monthly_income', 'credit_score', 'debt_to_income_ratio', 'financial_stress_level']:
    mask = np.random.rand(len(df_tracker_raw)) < 0.06
    df_tracker_raw[col] = df_tracker_raw[col].mask(mask)
# Giá trị vô lý cho điểm tín dụng
bad_idx = np.random.choice(df_tracker_raw.index, size = 15, replace = False)
df_tracker_raw.loc[bad_idx, 'credit_score'] = -999.0
```

Làm khuyết thiếu 6% các đặc trưng cốt lõi (monthly_income, credit_score,...) và thêm các giá trị bất hợp lý âm (-999.0) vào điểm tín dụng để thử thách độ bền vững của thuật toán.

Cuối cùng, hệ thống kết xuất đồng bộ ra 5 tệp thô mang hậu tố nhiễu (*_Noisy.csv) để làm tài nguyên đầu vào trực tiếp cho toàn bộ quy trình tiền xử lý tại mục sau.

```
# XUẤT CÁC FILE NHIỄU
df_income_raw.to_csv("Income_Noisy.csv", index = False)
df_8000_raw.to_csv("8000_Extended_Noisy.csv", index = False)
df_budget_raw.to_csv("Budget_Noisy.csv", index = False)
df_tracker_raw.to_csv("Tracker_Noisy.csv", index = False)
df_expenses_raw.to_csv("Expenses_Noisy.csv", index = False)
```

2.3. PHÂN TÍCH KHÁM PHÁ (EDA) & XÁC ĐỊNH LỖ HỔNG DỮ LIỆU

2.3.1. Khởi tạo và nạp nguồn dữ liệu mô phỏng nhiễu

Hệ thống triển khai hàm `pd.read_csv()` để nạp đồng thời 5 tệp dữ liệu chứa sai số hệ thống cấu hình ở chương trước (8000_Extended_Noisy.csv, Tracker_Noisy.csv, Budget_Noisy.csv, Expenses_Noisy.csv, Income_Noisy.csv). Bước này thiết lập trạng thái bộ nhớ ban đầu cho các biến đại diện: `df_8000`, `df_tracker`, `df_budget`, `df_expenses` và `df_income`.

```
df_8000 = pd.read_csv("8000_Extended_Noisy.csv")
df_tracker = pd.read_csv("Tracker_Noisy.csv")
df_budget = pd.read_csv("Budget_Noisy.csv")
df_expenses = pd.read_csv("Expenses_Noisy.csv")
df_income = pd.read_csv("Income_Noisy.csv")
```


2.3.2. Kiểm toán quy mô và định dạng sơ đồ dữ liệu

Thực thi lệnh `.shape` kết hợp hàm cấu trúc `.info()` nhằm bóc tách diện rộng kích thước hình học và kiểu con của từng trường dữ liệu (bao gồm `float64`, `int64`, `object`). Qua đó, hệ thống xác lập quy mô dữ liệu vĩ mô (với tệp hành vi mở rộng lên tới 8.000 dòng) và hồ sơ chỉ số tài chính cá nhân toàn diện (25 thuộc tính đầu vào).

```
# Xem kích thước dữ liệu
print("\n KÍCH THƯỚC DỮ LIỆU")
print("8000:", df_8000.shape)
print("Tracker:", df_tracker.shape)
print("Budget:", df_budget.shape)
print("Expenses:", df_expenses.shape)
print("Income:", df_income.shape)
print("-" * 70)
```

```
| KÍCH THƯỚC DỮ LIỆU
8000: (8000, 15)
Tracker: (3000, 25)
Budget: (3608, 3)
Expenses: (938, 6)
Income: (349, 6)
```

```
# Xem thông tin dữ liệu
print("\n THÔNG TIN DỮ LIỆU")
df_8000.info()
df_tracker.info()
df_budget.info()
df_expenses.info()
df_income.info()
print("-" * 70)
```

THÔNG TIN DỮ LIỆU

<class 'pandas.DataFrame'>

RangeIndex: 8000 entries, 0 to 7999

Data columns (total 15 columns):

#	Column	Non-Null Count	Dtype
0	Date	8000 non-null	str
1	Description	8000 non-null	str
2	Amount	7596 non-null	float64
3	Category	7695 non-null	str
4	PaymentMethod	8000 non-null	str
5	Location	8000 non-null	str
6	AccountType	8000 non-null	str
7	TransactionType	8000 non-null	str
8	DeviceUsed	8000 non-null	str
9	Currency	8000 non-null	str
10	MerchantType	8000 non-null	str
11	LoyaltyProgram	8000 non-null	str
12	Weekday	8000 non-null	str
13	Month	8000 non-null	str
14	TimeOfDay	8000 non-null	str

dtypes: float64(1), str(14)

memory usage: 937.6 KB

```

33 <class 'pandas.DataFrame'>
34 RangeIndex: 3000 entries, 0 to 2999
35 Data columns (total 25 columns):
36 | #    Column                                Non-Null Count  Dtype
37 | ---  ---                                -
38 | 0    date                                3000 non-null   str
39 | 1    user_id                             3000 non-null   int64
40 | 2    monthly_income                       2820 non-null   float64
41 | 3    monthly_expense_total                3000 non-null   float64
42 | 4    savings_rate                         3000 non-null   float64
43 | 5    budget_goal                         3000 non-null   float64
44 | 6    financial_scenario                  3000 non-null   str
45 | 7    credit_score                        2817 non-null   float64
46 | 8    debt_to_income_ratio                 2813 non-null   float64
47 | 9    loan_payment                        3000 non-null   float64
48 | 10   investment_amount                   3000 non-null   float64
49 | 11   subscription_services               3000 non-null   int64
50 | 12   emergency_fund                     3000 non-null   float64
51 | 13   transaction_count                   3000 non-null   int64
52 | 14   fraud_flag                         3000 non-null   int64
53 | 15   discretionary_spending              3000 non-null   float64
54 | 16   essential_spending                  3000 non-null   float64
55 | 17   income_type                         3000 non-null   str
56 | 18   rent_or_mortgage                    3000 non-null   float64
57 | 19   category                             3000 non-null   str
58 | 20   cash_flow_status                    3000 non-null   str
59 | 21   financial_advice_score               3000 non-null   float64
60 | 22   financial_stress_level               2829 non-null   str
61 | 23   actual_savings                       3000 non-null   float64
62 | 24   savings_goal_met                     3000 non-null   int64
63 dtypes: float64(14), int64(5), str(6)
64 memory usage: 586.1 KB
65 <class 'pandas.DataFrame'>
66 RangeIndex: 3608 entries, 0 to 3607
67 Data columns (total 3 columns):
68 | #    Column      Non-Null Count  Dtype
69 | ---  ---
70 | 0    date          3608 non-null   str
71 | 1    category       3608 non-null   str
72 | 2    amount         3507 non-null   float64
73 dtypes: float64(1), str(2)
74 memory usage: 84.7 KB

```

```

75 <class 'pandas.DataFrame'>
76 RangeIndex: 938 entries, 0 to 937
77 Data columns (total 6 columns):
78 | #      Column      Non-Null Count  Dtype
79 | ---  -
80 | 0      date_time    938 non-null    str
81 | 1      category     900 non-null    str
82 | 2      account      938 non-null    str
83 | 3      amount        887 non-null    float64
84 | 4      currency     938 non-null    str
85 | 5      tags          938 non-null    str
86 dtypes: float64(1), str(5)
87 memory usage: 44.1 KB
88 <class 'pandas.DataFrame'>
89 RangeIndex: 349 entries, 0 to 348
90 Data columns (total 6 columns):
91 | #      Column      Non-Null Count  Dtype
92 | ---  -
93 | 0      date_time    349 non-null    str
94 | 1      category     330 non-null    str
95 | 2      account      349 non-null    str
96 | 3      amount        335 non-null    float64
97 | 4      currency     349 non-null    str
98 | 5      tags          349 non-null    str
99 dtypes: float64(1), str(5)
00 memory usage: 16.5 KB

```

2.3.3. Dò tìm và định lượng khoảng trống dữ liệu

Áp dụng tổ hợp lệnh `.isnull()` `.sum()` quét tự động trên toàn bộ các tệp dữ liệu đầu vào.

```

# Kiểm tra giá trị khuyết trống thiếu
print("\n GIÁ TRỊ KHUYẾT TRỐNG THIẾU")
print(f"File 8000: \n{df_8000.isnull().sum()}\n")
print(f"File Tracker: \n{df_tracker.isnull().sum()}\n")
print(f"File Budget: \n{df_budget.isnull().sum()}\n")
print(f"File Exp: \n{df_expenses.isnull().sum()}\n")
print(f"File Inc: \n{df_income.isnull().sum()}")
print("-" * 70)

```

Kết quả ghi nhận chính xác quy mô các lỗ hổng dữ liệu (giá trị khuyết thiếu NaN) xuất hiện diện rộng tại các cột số tiền (Amount, amount), danh mục phân

loại tiêu dùng (Category, category) và các đặc trưng tài chính cốt lõi của tệp Tracker do lỗi đồng bộ hệ thống.

```
GIÁ TRỊ KHUYẾT TRỐNG THIỂU
File 8000:
Date                0
Description          0
Amount              404
Category             305
PaymentMethod        0
Location             0
AccountType          0
TransactionType      0
DeviceUsed           0
Currency             0
MerchantType         0
LoyaltyProgram       0
Weekday              0
Month                0
TimeOfDay            0
dtype: int64
```

```
21
22 File Tracker:
23 date                0
24 user_id             0
25 monthly_income      180
26 monthly_expense_total 0
27 savings_rate        0
28 budget_goal         0
29 financial_scenario   0
30 credit_score        183
31 debt_to_income_ratio 187
32 loan_payment        0
33 investment_amount    0
34 subscription_services 0
35 emergency_fund       0
36 transaction_count    0
37 fraud_flag          0
38 discretionary_spending 0
39 essential_spending   0
40 income_type          0
41 rent_or_mortgage     0
42 category            0
43 cash_flow_status     0
44 financial_advice_score 0
45 financial_stress_level 171
46 actual_savings       0
47 savings_goal_met     0
48 dtype: int64
```

```

50     File Budget:
51     date          0
52     category      0
53     amount       101
54     dtype: int64
55
56     File Exp:
57     date_time      0
58     category      38
59     account        0
60     amount        51
61     currency       0
62     tags           0
63     dtype: int64
64
65     File Inc:
66     date_time      0
67     category      19
68     account        0
69     amount        14
70     currency       0
71     tags           0
72     dtype: int64

```

2.3.4. Rà soát tần suất trùng lặp và đo lường thống kê

Hệ thống gọi thuộc tính `.duplicated().sum()` đếm tổng số các bản ghi bị nhân bản tuyệt đối, cô lập tỷ lệ nhiều trùng lặp tập trung nhiều nhất tại tệp chi tiêu (`df_expenses`).

```

# Kiểm tra trùng lặp
print("\n DỮ LIỆU TRÙNG LẶP")
print(f"Trùng lặp file 8000: {df_8000.duplicated().sum()}")
print(f"Trùng lặp file Tracker: {df_tracker.duplicated().sum()}")
print(f"Trùng lặp file Budget: {df_budget.duplicated().sum()}")
print(f"Trùng lặp file Exp: {df_expenses.duplicated().sum()}")
print(f"Trùng lặp file Inc: {df_income.duplicated().sum()}")
print("-" * 70)

```

```
DỮ LIỆU TRÙNG LẬP
Trùng lặp file 8000: 0
Trùng lặp file Tracker: 0
Trùng lặp file Budget: 0
Trùng lặp file Exp: 130
Trùng lặp file Inc: 0
```

Đồng thời, lệnh `.describe().round(2)` được thực thi nhằm tính toán tự động các tham số thống kê mô tả (Mean, Std, Max, Min), bậc lệch hai bất thường lớn

```
# Xem thống kê mô tả dữ liệu
print("\n THỐNG KÊ MÔ TẢ")
display(df_8000.describe().round(2)) #Thống kê mô tả cột số tiền
display(df_tracker.describe().round(2)) #Thống kê mô tả các chỉ số tài chính
print("-" * 70)
```

Chỉ số vĩ mô (df_tracker): Giá trị nhỏ nhất (Min) của cột điểm tín dụng `credit_score` đạt mức âm bất hợp lý (-999.00).

	user_id	monthly_income	monthly_expense_total	savings_rate	budget_goal	credit_score	debt_to_income_ratio	loan_payment	investment_amount	subscription_services
count	3000.00	2820.00	3000.00	3000.00	3000.00	2817.00	2813.00	3000.00	3000.00	3000.00
mean	1498.70	3998.05	3011.68	0.23	2811.07	671.21	0.35	508.58	400.61	4.97
std	287.35	998.06	801.15	0.10	490.86	131.85	0.15	199.44	235.36	2.56
min	1000.00	685.28	159.21	0.05	1175.57	-999.00	0.10	0.00	0.00	1.00
25%	1248.75	3354.75	2473.20	0.14	2481.96	645.00	0.23	375.34	234.78	3.00
50%	1496.50	4002.89	3023.81	0.23	2822.28	679.00	0.35	508.57	388.08	5.00
75%	1749.00	4650.16	3555.50	0.31	3131.00	713.00	0.48	638.47	559.84	7.00
max	1999.00	7407.94	5853.20	0.40	4386.50	847.00	0.60	1176.88	1292.30	9.00

emergency_fund	transaction_count	fraud_flag	discretionary_spending	essential_spending	rent_or_mortgage	financial_advice_score	actual_savings	savings_goal_met
3000.00	3000.00	3000.00		3000.00	3000.00	3000.00	3000.00	3000.00
1005.35	59.53	0.02		500.81	2215.59	1211.90	50.07	1156.42
485.78	23.10	0.15		199.80	596.27	400.48	29.09	1039.60
0.00	20.00	0.00		0.00	1000.00	300.00	0.10	0.00
674.01	38.75	0.00		361.56	1807.24	933.53	25.10	144.96
1010.48	60.00	0.00		496.34	2215.70	1213.60	48.70	1032.81
1337.82	79.00	0.00		640.10	2624.68	1482.86	76.10	1821.51
2585.36	99.00	1.00		1158.74	4245.90	2614.62	100.00	6589.66

Dòng tiền giao dịch (df_8000): Biên độ lệch giữa Giá trị trung bình và Trung vị (Median) cực kỳ lớn, đồng thời giá trị lớn nhất (Max) tăng vọt đột biến phản ánh sự tồn tại của 50 giao dịch dị biệt (Outliers) phóng đại 100 lần.

THỐNG KÊ MÔ TẢ	
...	
	Amount
count	7596.00
mean	23701.48
std	283676.10
min	54.11
25%	924.62
50%	3308.08
75%	11355.96
max	13129482.00

2.3.5. Khai phá và cô lập các lỗi hỏng hệ thống

Hồ sơ lỗi chính tả danh mục (Category): Lệnh `.value_counts()` trên tệp `df_8000` chỉ ra cấu trúc bảng chữ gốc bị phá vỡ và phân rã thành các cụm từ chứa ký tự đặc biệt và lỗi chính tả bao gồm: `Travell_err` (thay cho `Travel`), `Grocerie$` (chứa ký tự lạ thay cho `Grocery`), và `Biills` (lỗi lặp ký tự đơn thay cho `Bills`).

```
# Kiểm tra các danh mục chi tiêu
print("\n CATEGORY BẤT THƯỜNG")
print(df_8000["Category"].value_counts())
print("-" * 70)
```

```

CATEGORY BẤT THƯỜNG
Category
Online Shopping      803
Entertainment        800
Food                 785
Healthcare           769
Electronics          766
Travell_err          764
Transport            761
Biills               754
Grocerie$            750
Clothing             743
Name: count, dtype: int64

```


Hồ sơ định lượng lỗi điểm tín dụng vô lý (credit_score): Thực thi bộ lọc điều kiện logic (df_tracker["credit_score"] < 0).sum() để quét toàn bộ tập dữ liệu, hệ thống phát hiện và đếm chính xác 15 bản ghi bị tiêm giá trị âm hệ thống cực đoan là -999.0. Đây là vùng giá trị phá vỡ hoàn toàn quy luật phân phối thực tế (thang điểm tín dụng chuẩn chỉ dao động từ 300 đến 850).

```
# Kiểm tra Credit Score bất thường
print("\n CREDIT SCORE BẤT THƯỜNG")
print("Số lượng:", (df_tracker["credit_score"] < 0).sum())
print("-" * 70)
```

```
CREDIT SCORE BẤT THƯỜNG
Số lượng: 15
```

Hồ sơ lỗi chuỗi mốc thời gian: Lệnh .head() trên trường date_time của hai tệp df_expenses và df_income chỉ ra các dòng dữ liệu bị mất cấu trúc mốc thời gian nghiêm trọng, biến đổi từ định dạng ngày tháng và bị nhận diện sai lệch dưới dạng chuỗi văn bản (Object) chứa mã lỗi "ERR_DATE_FORMAT_99".

```
# Khám phá dữ liệu ngày tháng
print("\n DỮ LIỆU NGÀY THÁNG")
print(df_expenses["date_time"].head())
print(df_income["date_time"].head())
```

```
DỮ LIỆU NGÀY THÁNG
0      2025-11-30 00:00:00
1      ERR_DATE_FORMAT_99
2      2025-11-27 00:00:00
3      2025-11-27 00:00:00
4      2025-11-27 00:00:00
Name: date_time, dtype: str
0      2025-11-29 00:00:00
1      2025-11-29 00:00:00
2      2025-11-29 00:00:00
3      2025-11-29 00:00:00
4      2025-11-28 00:00:00
Name: date_time, dtype: str
```

Việc hoàn thành quy trình khám phá dữ liệu (EDA) toàn diện trên 5 bước đã giúp nhóm phác họa trọn vẹn "bản đồ lỗ hổng" của tập dữ liệu thô. Toàn bộ các phát hiện thực chứng này sẽ được chuyển giao trực tiếp làm đầu vào cho mục kế tiếp để triển khai bộ lọc làm sạch triệt để trước khi nạp vào hệ quản trị cơ sở dữ liệu MySQL.

2.4. TIỀN XỬ LÝ VÀ LÀM SẠCH DỮ LIỆU

2.4.1. Chuẩn hóa và làm sạch khối dữ liệu giao dịch lẻ (*Expenses - Income - Budget*)

2.4.1.1. Triệt tiêu trùng lặp

```
# Xóa dòng trùng nhau
df_expenses.drop_duplicates(inplace = True)
```

Hệ thống xử lý trực diện tệp nhật ký chi tiêu bằng cách thực thi lệnh `df_expenses.drop_duplicates(inplace=True)`. Lệnh này quét toàn bộ các bản ghi bị trùng lặp \$100%\$ về cả mốc thời gian, số tiền và danh mục (hậu quả của lỗi lưu đúp dữ liệu khi mất kết nối mạng), chỉ giữ lại bản ghi đầu tiên (`keep='first'`) để trả lại số lượng phản ánh đúng thực tế tiêu dùng.

```
# Kiểm tra trùng lặp
print(f"Trùng lặp file Exp: {df_expenses.duplicated().sum()}")
```

```
Trùng lặp file Exp: 0
```

2.4.1.2. Khôi phục toàn vẹn chuỗi thời gian

Đối với các chuỗi ký tự lỗi "ERR_DATE_FORMAT_99" xuất hiện tại cột `date_time` của cả hai tệp `df_expenses` và `df_income`, hệ thống áp dụng hàm

```
# Định dạng ngày tháng
df_expenses['date_time'] = pd.to_datetime(df_expenses['date_time'], errors='coerce').ffill().bfill()
df_income['date_time'] = pd.to_datetime(df_income['date_time'], errors='coerce').ffill().bfill()
```

Tham số `errors='coerce'` sẽ lập tức ép các chuỗi lỗi văn bản về trạng thái khuyết thiếu (NaT). Ngay sau đó, tổ hợp lệnh liên tiếp `.ffill().bfill()` được kích hoạt để tự động điền mốc thời gian của dòng liền trước hoặc liền sau vào ô lỗi. Phương pháp này giúp khôi phục trục thời gian tuyến tính mà không phải xóa bỏ dòng giao dịch, bảo toàn tính toàn vẹn của dòng tiền.

```
0    2025-11-30
1    2025-11-30
2    2025-11-27
3    2025-11-27
5    2025-11-26
Name: date_time, dtype: datetime64[us]
0    2025-11-29
1    2025-11-29
2    2025-11-29
3    2025-11-29
4    2025-11-28
Name: date_time, dtype: datetime64[us]
```

2.4.1.3. Đồng bộ điền khuyết tho đặc trưng biến

Xử lý biến định lượng (amount/amount): Áp dụng hàm `.fillna(df_expenses['amount'].median())` để thay thế ô trống bằng giá trị trung vị (Median).

```
# Xử lý các giá trị bị khuyết trống thiếu ở cột Amount với giá trị ở giữa (Median) - ở cột Category với chữ xuất hiện nhiều nhất (Mode)
df_expenses['amount'] = df_expenses['amount'].fillna(df_expenses['amount'].median())
df_income['amount'] = df_income['amount'].fillna(df_income['amount'].median())
df_budget['amount'] = df_budget['amount'].fillna(df_budget['amount'].median())
```

Nhóm sử dụng trung vị thay vì trung bình cộng (Mean) để tránh việc giá trị điền khuyết bị kéo lệch bởi các giao dịch mua sắm đột xuất có giá trị quá cao hoặc quá thấp.

Xử lý biến định tính (category/category): Thực thi lệnh `.fillna(df_expenses['category'].mode()[0])` nhằm gán các giao dịch bị khuyết danh mục vào nhóm xuất hiện nhiều nhất (Yếu vị - Mode).

```
df_expenses['category'] = df_expenses['category'].fillna(df_expenses['category'].mode()[0])
df_income['category'] = df_income['category'].fillna(df_income['category'].mode()[0])
```

```
# Kiểm tra các giá trị khuyết thiếu
print("\n GIÁ TRỊ KHUYẾT TRỐNG THIẾU")
print(f"File Budget: \n{df_budget.isnull().sum()}\n")
print(f"File Exp: \n{df_expenses.isnull().sum()}\n")
print(f"File Inc: \n{df_income.isnull().sum()}")
```

```
GIÁ TRỊ KHUYẾT TRỐNG THIẾU
File Budget:
date          0
category      0
amount        0
dtype: int64

File Exp:
date_time     0
category      0
account        0
amount        0
currency      0
tags          0
dtype: int64

File Inc:
date_time     0
category      0
account        0
amount        0
currency      0
tags          0
dtype: int64
```

2.4.2. Xử lý giá trị phi lý và khử nhiễu ngoại lai bằng IQR trên tập Tracker

2.4.2.1. Hiệu chỉnh giá trị vô lý điểm tín dụng

Để dọn dẹp 15 lỗi hệ thống mang giá trị âm cực đoan -999.0 tại cột `credit_score`, hệ thống chạy lệnh

```
# credit_score = -999 -> NaN  
  
df_track["credit_score"] = df_track["credit_score"].replace(-999.0, np.nan)
```

Sau khi cô lập giá trị vô lý về dạng khuyết thiếu (NaN), hệ thống sử dụng vòng lặp tự động duyệt qua danh sách các cột số gồm `monthly_income`, `credit_score`, `debt_to_income_ratio` để đồng loạt lấp đầy khoảng trống bằng hàm toán học `.median()`

```
# 3. Điền giá trị thiếu  
  
numeric_cols = [  
    "monthly_income",  
    "credit_score",  
    "debt_to_income_ratio"  
]  
  
for col in numeric_cols:  
    df_track[col] = df_track[col].fillna(df_track[col].median())
```

Riêng biến phân cấp áp lực tài chính `financial_stress_level` được lấp đầy bằng phương thức `.mode()[0]`.

```
df_track["financial_stress_level"] = (  
    df_track["financial_stress_level"]  
    .fillna(df_track["financial_stress_level"].mode()[0])  
)
```

2.4.2.2. Toán học IQR khử ngoại lai thu nhập

Nhằm triệt tiêu hiện tượng nhiễu đồ thị do các điểm thu nhập quá cao hoặc quá thấp phá vỡ phân phối chuẩn, hệ thống thiết lập bộ lọc khoảng tứ phân vị (Interquartile Range) trên biến `monthly_income`

Xác lập công thức toán học tính biên giới hạn

$$IQR = Q3 - Q1$$

$$\text{lower_bound} = Q1 - 1.5 * IQR$$

$$\text{upper_bound} = Q3 + 1.5 * IQR$$

```
# 4. Xử lý Outlier bằng IQR
# (Thay bằng Median)

Q1 = df_track["monthly_income"].quantile(0.25)
Q3 = df_track["monthly_income"].quantile(0.75)
IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

median_income = df_track["monthly_income"].median()
```

Trong đó, Q1 đại diện cho giá trị bách phân vị thứ 25 (ngưỡng mà tại đó 25% dữ liệu có giá trị nhỏ hơn hoặc bằng nó) và Q3 đại diện cho giá trị bách phân vị thứ 75 của phân phối biến thu nhập hàng tháng (monthly_income). Khoảng tứ phân vị IQR đo lường độ phân tán của 50% số lượng quan sát ở trung tâm phân phối. Hệ số tiêu chuẩn 1.5 là hằng số tối ưu do John Tukey đề xuất nhằm thiết lập ranh giới loại bỏ nhiễu mà không làm tổn thương các điểm dữ liệu thuộc phân phối đuôi dài thực tế.

Thông qua việc tính toán các ngưỡng này, hệ thống thực hiện xác định và đếm chính xác số lượng bản ghi dị biệt nằm ngoài phân phối an toàn bằng biểu thức điều kiện logic

```
median_income = df_track["monthly_income"].median()

outlier_count = (
    (df_track["monthly_income"] < lower) |
    (df_track["monthly_income"] > upper)
).sum()

print("\nSố Outlier của monthly_income:", outlier_count)
```

Để tránh việc các giá trị cực đoan này làm lệch ma trận hiệp phương sai và làm sai lệch hàm mất mát khi huấn luyện các mô hình học máy (như cấu trúc tuyến tính của Logistic Regression hay việc phân nhánh của Decision Tree), hệ thống triển khai phương thức lọc nâng cao `.loc[]` để tiếp cận trực tiếp các tọa độ dòng bị lỗi và gán đè bằng giá trị trung vị mẫu (`median_income`):

```
df_track.loc[
    (df_track["monthly_income"] < lower) |
    (df_track["monthly_income"] > upper),
    "monthly_income"
] = median_income
```

Việc gán đè bằng giá trị trung vị (Median Imputation) là một kỹ thuật làm sạch có tính bền vững cao (Robust statistic). Nó giúp giữ lại cấu trúc mẫu dòng tiền tổng thể của người dùng mà không làm thay đổi xu hướng hội tụ trung tâm, đồng thời triệt tiêu hoàn toàn độ nhọn bất thường của phân phối dữ liệu thô

2.4.2.3. Kiểm toán và xác thực cấu trúc sau tiền xử lý

Sau khi hoàn tất chu trình dọn dẹp, hệ thống tái kiểm tra tính toàn vẹn của tập dữ liệu `df_track` bằng tổ hợp lệnh `.isnull().sum()` và `.describe(include="all")`.

```
# 5. Kiểm tra sau khi clean

print("\n===== GIÁ TRỊ THIẾU =====")
print(df_track.isnull().sum())

print("\n===== THỐNG KÊ =====")
print(df_track.describe(include="all"))

print("\n===== THÔNG TIN =====")
print(df_track.info())
```

Tính toàn vẹn dữ liệu đạt mức tuyệt đối: Kết quả từ hàm kiểm tra khuyết thiếu `df_track.isnull().sum()` ghi nhận giá trị bằng 0 đồng nhất trên tất cả 25 trường thuộc tính. Đồng thời, cấu trúc thông tin tổng quát xác nhận chỉ số Non-Null Count đạt ngưỡng 3000 non-null trên toàn bộ các cột dữ liệu. Điều này minh chứng tất cả lỗi hỏng khuyết thiếu ngẫu nhiên ban đầu đã được lấp đầy hoàn chỉnh

```
===== GIÁ TRỊ THIẾU =====
date                                0
user_id                             0
monthly_income                     0
monthly_expense_total               0
savings_rate                        0
budget_goal                         0
financial_scenario                  0
credit_score                        0
debt_to_income_ratio               0
loan_payment                       0
investment_amount                   0
subscription_services              0
emergency_fund                     0
transaction_count                  0
fraud_flag                         0
discretionary_spending              0
essential_spending                  0
income_type                         0
rent_or_mortgage                    0
category                           0
cash_flow_status                    0
financial_advice_score              0
financial_stress_level              0
actual_savings                      0
savings_goal_met                    0
dtype: int64
```



```

===== THÔNG TIN =====
<class 'pandas.DataFrame'>
RangeIndex: 3000 entries, 0 to 2999
Data columns (total 25 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   date                                  3000 non-null   str
1   user_id                              3000 non-null   int64
2   monthly_income                       3000 non-null   float64
3   monthly_expense_total                3000 non-null   float64
4   savings_rate                         3000 non-null   float64
5   budget_goal                          3000 non-null   float64
6   financial_scenario                   3000 non-null   str
7   credit_score                         3000 non-null   float64
8   debt_to_income_ratio                 3000 non-null   float64
9   loan_payment                         3000 non-null   float64
10  investment_amount                    3000 non-null   float64
11  subscription_services                3000 non-null   int64
12  emergency_fund                       3000 non-null   float64
13  transaction_count                    3000 non-null   int64
14  fraud_flag                           3000 non-null   int64
15  discretionary_spending                3000 non-null   float64
16  essential_spending                   3000 non-null   float64
17  income_type                           3000 non-null   str
18  rent_or_mortgage                     3000 non-null   float64
19  category                             3000 non-null   str
20  cash_flow_status                     3000 non-null   str
21  financial_advice_score                3000 non-null   float64
22  financial_stress_level                3000 non-null   str
23  actual_savings                       3000 non-null   float64
24  savings_goal_met                      3000 non-null   int64
dtypes: float64(14), int64(5), str(6)
memory usage: 586.1 KB
None

```

Khôi phục tính hợp lý nghiệp vụ tài chính: Tại cột đặc trưng điểm tín dụng (credit_score) ở giá trị nhỏ nhất (min) ghi nhận đạt mức hợp lý là 515.000000, điểm trung vị (50%) đạt 679.000000 và điểm tối đa (max) đạt 847.000000. Kết quả này xác thực toàn bộ các nhãn nhiễu hệ thống -999.0 đã bị cô lập và loại bỏ thành công, đưa dải điểm tín dụng quay về đúng quỹ đạo phân phối chuẩn mực của hệ thống tài chính thực tế.

```

30  ===== THỐNG KÊ =====
31  |           |           date           user_id  monthly_income  monthly_expense_total  \
32  count      3000  3000.000000    3000.000000          3000.000000
33  unique      60           NaN           NaN           NaN
34  top      2019-01-01           NaN           NaN           NaN
35  freq       50           NaN           NaN           NaN
36  mean      NaN  1498.699000    3998.343117    3011.682230
37  std      NaN   287.352782     967.647641     801.151864
38  min      NaN  1000.000000     685.280000     159.210000
39  25%      NaN  1248.750000    3402.242500    2473.205000
40  50%      NaN  1496.500000    4002.890000    3023.810000
41  75%      NaN  1749.000000    4600.605000    3555.502500
42  max      NaN  1999.000000    7407.940000    5853.200000
43
44  |           | savings_rate  budget_goal  financial_scenario  credit_score  \
45  count      3000.000000  3000.000000          3000    3000.000000
46  unique      NaN           NaN           3           NaN
47  top      NaN           NaN           normal           NaN
48  freq      NaN           NaN           1739           NaN
49  mean      0.225897  2811.070463           NaN    680.075333
50  std      0.101417   490.855344           NaN    47.917885
51  min      0.050000  1175.570000           NaN    515.000000
52  25%      0.140000  2481.957500           NaN    649.000000
53  50%      0.230000  2822.285000           NaN    679.000000
54  75%      0.310000  3131.000000           NaN    710.000000
55  max      0.400000  4386.500000           NaN    847.000000
56
57  |           | debt_to_income_ratio  loan_payment  ...  discretionary_spending  \
58  count      3000.000000  3000.000000  ...          3000.000000
59  unique      NaN           NaN  ...           NaN
60  top      NaN           NaN  ...           NaN
61  freq      NaN           NaN  ...           NaN
62  mean      0.351263   508.58113  ...          500.810990
63  std      0.140820   199.44458  ...          199.800377
64  min      0.100000    0.00000  ...           0.000000
65  25%      0.230000   375.34000  ...          361.562500
66  50%      0.350000   508.57000  ...          496.340000
67  75%      0.470000   638.47000  ...          640.105000
68  max      0.600000  1176.88000  ...          1158.740000
69

```

Hội tụ phân phối thu nhập và kiểm soát ngoại lai: Sau khi áp dụng thuật toán hàng rào Tukey gán đề các ngoại lai cực đoan về trung vị mẫu, biến thu nhập hàng tháng (monthly_income) ghi nhận giá trị lớn nhất (max) được kiểm soát an toàn ở mức 7,407.940000, trong khi giá trị nhỏ nhất (min) giữ ở mức 685.280000. Độ

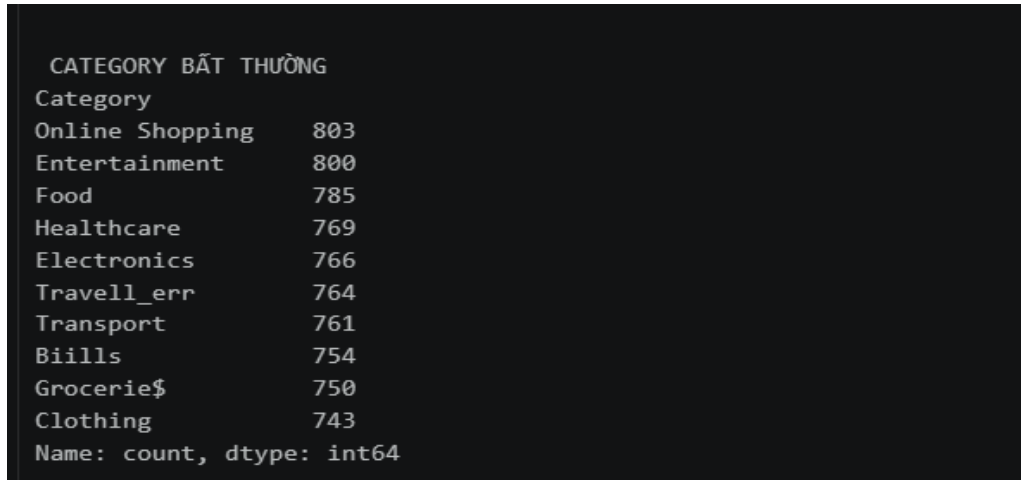
lệch chuẩn (std) của đặc trưng này đạt mức ổn định 967.64, giúp phân phối thu nhập loại bỏ hoàn toàn hiện tượng lệch phải cực đoan, tạo điều kiện tối ưu cho sự hội tụ của các thuật toán học máy phía sau.

70		essential_spending	income_type	rent_or_mortgage	category \
71	count	3000.000000	3000	3000.000000	3000
72	unique	NaN	3	NaN	10
73	top	NaN	Salary	NaN	Insurance
74	freq	NaN	2154	NaN	321
75	mean	2215.587100	NaN	1211.897797	NaN
76	std	596.267983	NaN	400.482024	NaN
77	min	1000.000000	NaN	300.000000	NaN
78	25%	1807.235000	NaN	933.530000	NaN
79	50%	2215.705000	NaN	1213.605000	NaN
80	75%	2624.675000	NaN	1482.860000	NaN
81	max	4245.900000	NaN	2614.620000	NaN
82					
83		cash_flow_status	financial_advice_score	financial_stress_level \	
84	count	3000	3000.000000		3000
85	unique	3	NaN		3
86	top	Positive	NaN		Low
87	freq	1783	NaN		1598
88	mean	NaN	50.071133		NaN
89	std	NaN	29.087360		NaN
90	min	NaN	0.100000		NaN
91	25%	NaN	25.100000		NaN
92	50%	NaN	48.700000		NaN
93	75%	NaN	76.100000		NaN
94	max	NaN	100.000000		NaN
95					
96		actual_savings	savings_goal_met		
97	count	3000.000000	3000.000000		
98	unique	NaN	NaN		
99	top	NaN	NaN		
00	freq	NaN	NaN		
01	mean	1156.418457	0.092333		
02	std	1039.597384	0.289544		
03	min	0.000000	0.000000		
04	25%	144.965000	0.000000		
05	50%	1032.810000	0.000000		
06	75%	1821.510000	0.000000		
07	max	6589.660000	1.000000		
08					
09	[11 rows x 25 columns]				

2.4.3. Xử lý nhiễu diện rộng và đồng bộ từ vựng trên Tập hành vi mở rộng 8000

2.4.3.1. Ánh xạ từ điển và xử lý bất đồng bộ từ vựng danh mục

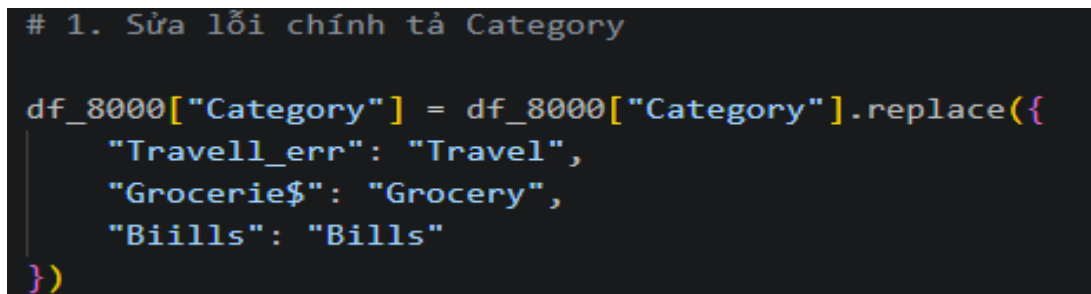
Tại trường thuộc tính định tính Category, sự xuất hiện của các nhãn sai chính tả do lỗi nhập liệu hệ thống như Travell_err, Grocerie\$ hay Biills làm phân rã các danh mục chi tiêu, gây ra hiện tượng bùng nổ chiều dữ liệu giả tạo khi mã hóa biến số.



Category	Count
Online Shopping	803
Entertainment	800
Food	785
Healthcare	769
Electronics	766
Travell_err	764
Transport	761
Biills	754
Grocerie\$	750
Clothing	743

Name: count, dtype: int64

Hệ thống đã triển khai phương thức .replace() thông qua một từ điển ánh xạ cấu trúc chuẩn để đồng bộ hóa từ vựng:



```
# 1. Sửa lỗi chính tả Category

df_8000["Category"] = df_8000["Category"].replace({
    "Travell_err": "Travel",
    "Grocerie$": "Grocery",
    "Biills": "Bills"
})
```

Việc loại bỏ các ký tự nhiễu (như dấu \$) và sửa lỗi chính tả giúp đưa các danh mục về đúng 3 nhóm nhãn chuẩn (Travel, Grocery, Bills). Quy trình này bảo toàn tính toàn vẹn ngữ nghĩa, đảm bảo các thuật toán phân nhóm hành vi phía sau không bị phân tán trọng số bởi các nhãn nhiễu.

Category	
Online Shopping	1108
Entertainment	800
Food	785
Healthcare	769
Electronics	766
Travel	764
Transport	761
Bills	754
Grocery	750
Clothing	743
Name: count, dtype: int64	

2.4.3.2. Khôi phục dữ liệu khuyết thiếu trên tập quy mô lớn

Đối với thuộc tính phân lớp (Category): Áp dụng giải thuật điền khuyết bằng giá trị yếu vị. Những bản ghi khuyết thiếu danh mục sẽ được tự động lấp đầy bằng nhãn có tần suất xuất hiện cao nhất trong tập dữ liệu

```
# Category -> thay bằng giá trị xuất hiện nhiều nhất
df_8000["Category"] = df_8000["Category"].fillna(
    df_8000["Category"].mode()[0]
```

Đối với thuộc tính định lượng (Amount): Do đặc trưng số tiền chi tiêu thường có xu hướng phân phối lệch phải và chịu tác động mạnh bởi các giao dịch lớn, hệ thống sử dụng giá trị trung vị mẫu làm tham số lấp đầy ô trống, đảm bảo giá trị điền khuyết không bị kéo theo các điểm cực đoan.

```
# Amount -> thay bằng trung vị
df_8000["Amount"] = df_8000["Amount"].fillna(
    df_8000["Amount"].median()
)
```

2.4.3.3. Cô lập và chế tài ngoại lai bằng hàng rào toán học IQR

Để triệt tiêu ảnh hưởng của 50 bản ghi bị phóng đại dòng tiền lên 100 lần (nhiều mô phỏng hệ thống), giải thuật hàng rào IQR tiếp tục được tái cấu trúc dành riêng cho biến Amount:

$$IQR = Q_3 - Q_1$$

$$\text{Lower Bound} = Q_1 - 1.5 * IQR; \text{Upper Bound} = Q_3 + 1.5 * IQR$$

```
# 3. Xử lý Outlier Amount

Q1 = df_8000["Amount"].quantile(0.25)
Q3 = df_8000["Amount"].quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

median_amount = df_8000["Amount"].median()

df_8000.loc[
    (df_8000["Amount"] < lower_bound) |
    (df_8000["Amount"] > upper_bound),
    "Amount"
] = median_amount
```

Bằng cách áp dụng toán tử .loc[] kết hợp với điều kiện logic, tất cả các giao dịch vượt ngưỡng biên trên (do bị nhân 100 lần) hoặc biên dưới phi thực tế đều bị ép gán về mức trung vị mẫu median_amount. Phương pháp này giúp phân phối của biến Amount co về dải giá trị tiêu dùng thực tế một cách mượt mà, triệt tiêu hoàn toàn hiện tượng phương sai ngoại lai làm lệch các mô hình ước lượng khoảng cách.

Date	0
Description	0
Amount	0
Category	0
PaymentMethod	0
Location	0
AccountType	0
TransactionType	0
DeviceUsed	0
Currency	0
MerchantType	0
LoyaltyProgram	0
Weekday	0
Month	0
TimeOfDay	0
dtype:	int64

2.5. ĐỒNG BỘ VÀ LƯU TRỮ KHO DỮ LIỆU SẠCH VÀO MYSQL

2.5.1.1. Cơ chế nạp dữ liệu tự động (ETL)

Quy trình được tối ưu hóa thông qua lớp đối tượng Connection_DB bằng cách kết hợp thư viện mysql.connector và SQLAlchemy:

- **Kết nối an toàn:** Cấu hình tài khoản (root) được cô lập trong khối lệnh try - except - finally giúp tự động đóng cổng kết nối an toàn sau khi hoàn tác, giải phóng tài nguyên hệ thống.
- **Tiền lọc cấu trúc:** Trước khi nạp, hàm .dropna(how='all') và .drop_duplicates() được thực thi nhằm triệt tiêu hoàn toàn các dòng trống hoặc các bản ghi trùng lặp cơ học phát sinh khi ghi file.
- **Ánh xạ tự động:** Hàm to_sql() với tham số if_exists='replace' tự động cấu trúc lại các bảng dữ liệu trong MySQL, đồng bộ hóa kiểu dữ liệu từ Python sang SQL.

```
import pandas as pd
import mysql.connector
from mysql.connector import Error
from sqlalchemy import create_engine

class Connection_DB:
    def __init__(self):
        # Cấu hình tài khoản theo mẫu
        self.db_config = {
            'host': 'localhost',
            'user': 'root',
            'password': '1234',
            'database': 'finance_db'
        }

    def connect_db(self):
        connection = None
        try:
            connection = mysql.connector.connect(**self.db_config)
            if connection.is_connected():
                print("Kết nối MySQL thành công!")
                datasets = {
                    "Income_CLEANED": "Income_CLEANED.csv",
                    "Expenses_CLEANED": "Expenses_CLEANED.csv",
                    "Budget_CLEANED": "Budget_CLEANED.csv",
                    "Tracker_CLEANED": "Tracker_CLEANED.csv",
                    "8000_Extended_Clean": "8000_Extended_Clean.csv"
                }
                engine = create_engine(f'mysql+mysqlconnector://{self.db_config["user"]}:{self.db_config["password"]}@{self.db_config["host"]}/{self.db_config["database"]}')
                # Quét trùng và nạp bảng dữ liệu nhanh
                for table_name, file_path in datasets.items():
                    df = pd.read_csv(file_path).dropna(how='all').drop_duplicates()
                    df.to_sql(name=table_name, con=engine, if_exists='replace', index=False)
        except Error as e:
            print(f"Lỗi: {e}")
        finally:
            if connection and connection.is_connected():
                connection.close()
                print("Đã ngắt kết nối an toàn.")

if __name__ == "__main__":
    db = Connection_DB()
    db.connect_db()
```

MySQL Workbench

BaoCao - Vietnam - not supp...

File Edit View Query Database Server Tools Scripting Help

8000_extended_clean budget_cleaned expenses_cleaned income_cleaned tracker_cleaned

1 * SELECT * FROM finance_db.tracker_cleaned

SQL Additions

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

Result Grid

date	user_id	monthly_income	monthly_expense_total	savings_rate	budget_goal	financial_scenario	credit_score	debt_to_income_ratio	loan_payment	investment_amount	subscription_services	emergency_fund	transaction_count	fraud_flag	discretionary_spending	etc
2019-01-01	1584	3119.58	3212.07	0.38	3676.11	inflation	723	0.56	125.77	699.22	3	510.58	68	0	837.55	1518
2019-01-31	1045	3262.44	3732.81	0.1	2607.17	inflation	670	0.42	454.19	360.34	4	1154.41	41	0	534.51	3168
2019-02-02	1756	2931.2	3335.58	0.15	3004.14	inflation	691	0.24	971.82	0	5	1433.02	90	0	353.67	1504
2019-02-01	1724	3360.79	2327.59	0.17	3346.67	normal	717	0.16	482.76	182.06	5	227.37	94	0	594.08	1400
2019-02-01	1600	4606.87	2182.58	0.34	2670.09	inflation	795	0.25	263.74	342.78	9	589.81	73	0	556.86	1000
2019-02-31	1985	4628.39	3876.57	0.26	2079.72	recession	643	0.43	323.2	451.11	8	1131.03	56	0	431.04	2330
2019-02-29	1194	4145.45	3035.61	0.15	2215.8	normal	679	0.23	609.88	720.42	8	995.12	69	0	442.5	2860
2019-02-30	1600	3202.37	2689.51	0.25	3650.48	normal	655	0.24	524.64	224.15	4	732.71	65	0	602.88	3536
2019-02-29	1835	4765.11	2628.56	0.06	2813.81	normal	715	0.35	103.3	0	7	1623.05	99	0	733.67	2503
2019-02-28	1454	3381.79	3370.8	0.13	2195.92	normal	749	0.35	643.48	261.39	9	1037.1	91	0	464.34	1691
2019-02-28	1641	5811.71	1716.77	0.11	1176.17	normal	611	0.36	116.14	461.66	6	1005.61	71	0	464.34	1691

8000_extended_clean 1

Output

#	Time	Action	Message	Duration / Fetch
1	22:08:52	SELECT * FROM finance_db.8000_extended_clean LIMIT 0, 1000	1000 row(s) returned	0.000 sec / 0.000 sec
2	22:09:12	SELECT * FROM finance_db.budget_cleaned LIMIT 0, 1000	1000 row(s) returned	0.016 sec / 0.000 sec
3	22:09:42	SELECT * FROM finance_db.expenses_cleaned LIMIT 0, 1000	793 row(s) returned	0.016 sec / 0.000 sec
4	22:10:01	SELECT * FROM finance_db.income_cleaned LIMIT 0, 1000	343 row(s) returned	0.000 sec / 0.000 sec
5	22:10:17	SELECT * FROM finance_db.tracker_cleaned LIMIT 0, 1000	1000 row(s) returned	0.000 sec / 0.015 sec

MySQL Workbench

BarCao - Warning - not supp...

File Edit View Query Database Server Tools Scripting Help

8000_extended_clean budget_cleaned expenses_cleaned income_cleaned tracker_cleaned

1 SELECT * FROM finance_db.income_cleaned;

Result Grid

Date_time	Category	account	amount	currency	tags
2025-11-29	Job	acct_1	49	BYN	tag_1
2025-11-29	Job	acct_1	18	BYN	tag_2
2025-11-29	Job	acct_1	32	BYN	tag_3
2025-11-29	Job	acct_1	109	BYN	tag_4
2025-11-29	Second work	acct_1	132	BYN	2th_work
2025-11-29	Debt return / Borrowed money	acct_1	49	BYN	tag_5
2025-11-29	Job	acct_1	18	BYN	tag_1
2025-11-22	Job	acct_1	44	BYN	tag_1
2025-11-22	Job	acct_1	10	BYN	tag_2
2025-11-22	Job	acct_1	11	BYN	tag_3
2025-11-20	Job	acct_1	36	BYN	tag_4
2025-11-18	Job	acct_1	17	BYN	tag_1

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	22:08:52	SELECT * FROM finance_db.8000_extended_clean LIMIT 0, 1000	1000 row(s) returned	0.000 sec / 0.000 sec
2	22:09:12	SELECT * FROM finance_db.budget_cleaned LIMIT 0, 1000	1000 row(s) returned	0.016 sec / 0.000 sec
3	22:09:42	SELECT * FROM finance_db.expenses_cleaned LIMIT 0, 1000	799 row(s) returned	0.016 sec / 0.000 sec
4	22:10:01	SELECT * FROM finance_db.income_cleaned LIMIT 0, 1000	349 row(s) returned	0.000 sec / 0.000 sec

SQL Additions

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

MySQL Workbench

BarCao - Warning - not supp...

File Edit View Query Database Server Tools Scripting Help

8000_extended_clean budget_cleaned expenses_cleaned income_cleaned tracker_cleaned

1 SELECT * FROM finance_db.expenses_cleaned;

Result Grid

Date_time	Category	account	amount	currency	tags
2025-11-30	Health	acct_1	114	BYN	tag_1
2025-11-30	Food	acct_1	5	BYN	tag_1
2025-11-27	Public transport	acct_2	1	BYN	tag_1
2025-11-27	Cafe	acct_1	10	BYN	tag_1
2025-11-26	Cafe	acct_3	2	BYN	tag_1
2025-11-26	Public transport	acct_2	1	BYN	tag_1
2025-11-25	Public transport	acct_2	1	BYN	tag_1
2025-11-25	Cafe	acct_1	11	BYN	tag_1
2025-11-25	Taxi	acct_1	4	BYN	tag_1
2025-11-24	Food	acct_1	9	BYN	tag_1
2025-11-24	Public transport	acct_2	1	BYN	tag_1
2025-11-18	Cafe	acct_1	4	BYN	tag_1

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	22:08:52	SELECT * FROM finance_db.8000_extended_clean LIMIT 0, 1000	1000 row(s) returned	0.000 sec / 0.000 sec
2	22:09:12	SELECT * FROM finance_db.budget_cleaned LIMIT 0, 1000	1000 row(s) returned	0.016 sec / 0.000 sec
3	22:09:42	SELECT * FROM finance_db.expenses_cleaned LIMIT 0, 1000	799 row(s) returned	0.016 sec / 0.000 sec

SQL Additions

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

MySQL Workbench - BacCas - Warning - not supp. - x

File Edit View Query Database Server Tools Scripting Help

1 * SELECT * FROM finance_db.budget_cleaned;

SQL Additions

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

Result Grid

Date	category	amount
2022-07-06 05:57:10 +0000	Restaurant	5.5
2022-07-06 05:57:27 +0000	Market	2
2022-07-06 05:58:12 +0000	Coffee	30.1
2022-07-06 05:58:25 +0000	Market	17.33
2022-07-06 05:59:00 +0000	Restaurant	5.5
2022-07-06 05:59:13 +0000	Market	11.78
2022-07-06 05:59:41 +0000	Restaurant	10
2022-07-06 05:59:44 +0000	Market	1.38
2022-07-06 05:59:45 +0000	Restaurant	10
2022-07-06 05:59:56 +0000	Market	2.73
2022-07-06 06:00:04 +0000	Market	1.94
2022-07-06 06:00:13 +0000	Transport	6

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	22:08:52	SELECT * FROM finance_db.budget_cleaned LIMIT 0, 1000	1000 row(s) returned	0.000 sec / 0.000 sec
2	22:09:12	SELECT * FROM finance_db.budget_cleaned LIMIT 0, 1000	1000 row(s) returned	0.016 sec / 0.000 sec

MySQL Workbench - BacCas - Warning - not supp. - x

File Edit View Query Database Server Tools Scripting Help

1 * SELECT * FROM finance_db.budget_extended_clean;

SQL Additions

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

Result Grid

Date	Description	Amount	Category	PaymentMethod	Location	AccountType	TransactionType	DeviceUsed	Currency	MerchantType	LoyaltyProgram	Weekday	Month	TimeOfDay
2025-04-05	Transaction at BCTC	20466.49	Travel	Debit Card	Kolkata	Salary	Debit	Mobile	INR	Service	No	Saturday	April	Night
2025-12-06	Transaction at Myntia	1919.05	Online Shopping	Net Banking	Hyderabad	Salary	Debit	Desktop	INR	Online Store	Yes	Saturday	December	Evening
2025-06-20	Transaction at BigBazaar	3308.078	Grocery	Debit Card	Bangalore	Savings	Debit	Mobile	INR	Retail	No	Friday	June	Evening
2025-04-23	Transaction at Property Tax Online	1071.04	Bills	Net Banking	Delhi	Current	Debit	POS Terminal	INR	Service	Yes	Wednesday	April	Afternoon
2025-03-06	Transaction at Axa Store	10722.8	Electronics	Debit Card	Mumbai	Current	Debit	Mobile	INR	Retail	Yes	Thursday	March	Evening
2025-05-30	Transaction at Max Fashion	3502.77	Clothing	Debit Card	Bangalore	Savings	Debit	POS Terminal	INR	Retail	No	Thursday	October	Morning
2025-06-11	Transaction at Property Tax Online	5334.34	Bills	Credit Card	Pune	Current	Debit	Desktop	INR	Service	Yes	Wednesday	June	Morning
2025-01-03	Transaction at Airtel	1472.58	Bills	UPI	Mumbai	Salary	Debit	Desktop	INR	Service	Yes	Friday	January	Morning
2025-07-27	Transaction at Wipro	1963.86	Clothing	Credit Card	Delhi	Current	Debit	POS Terminal	INR	Retail	No	Sunday	July	Afternoon
2025-09-16	Transaction at Axa	250.69	Entertainment	Credit Card	Chennai	Salary	Debit	POS Terminal	INR	Subscription	No	Saturday	August	Morning
2025-03-04	Transaction at Airtel	3989.88	Bills	Debit Card	Bangalore	Savings	Debit	Desktop	INR	Service	Yes	Tuesday	March	Evening
1976262.115	Transaction at Capital China	435.04	Road	Credit Card	Mumbai	Customer	Debit	POS Terminal	INR	Subscription	Yes	Monday	June	Afternoon

Output

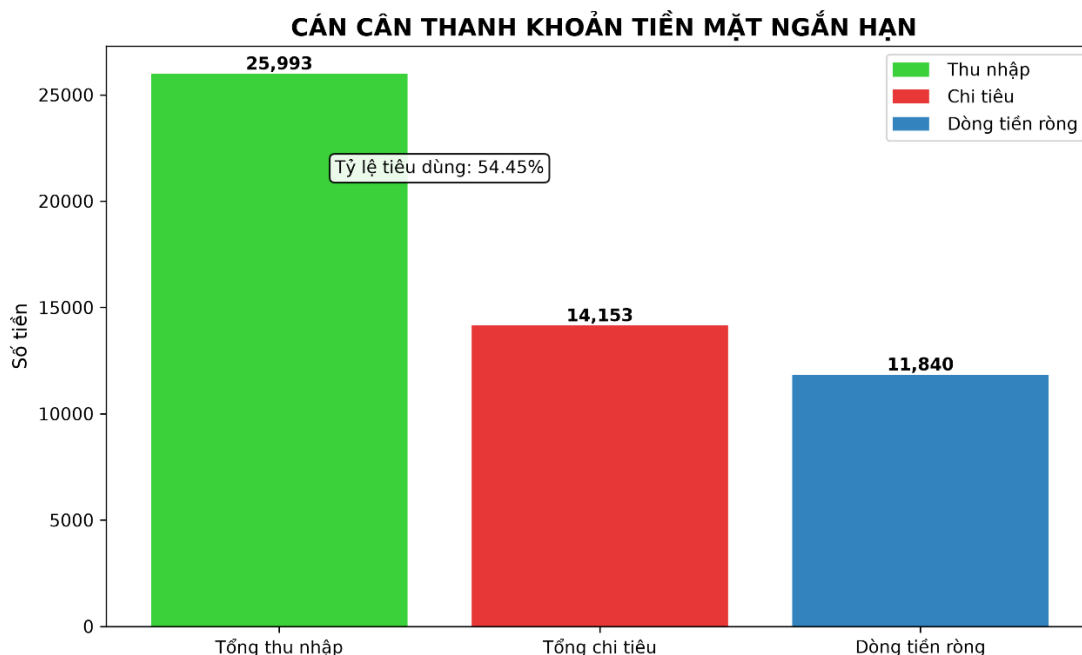
Action Output

#	Time	Action	Message	Duration / Fetch
1	22:08:52	SELECT * FROM finance_db.budget_extended_clean LIMIT 0, 1000	1000 row(s) returned	0.000 sec / 0.000 sec

CHƯƠNG 3: PHÂN TÍCH KHÁM PHÁ DỮ LIỆU TRỰC QUAN

3.1. CÁN CÂN THANH KHOẢN TIỀN MẶT NGẮN HẠN

```
66 # =====
67 # 1. CÁN CÂN THANH KHOẢN TIỀN MẶT NGẮN HẠN
68 # Tính tổng thu nhập bằng cách cộng toàn bộ cột amount trong bảng income.
69 total_income = income["amount"].sum()
70 # Tính tổng chi tiêu bằng cách cộng toàn bộ cột amount trong bảng expenses.
71 total_expenses = expenses["amount"].sum()
72 # Tính dòng tiền ròng = tổng thu nhập - tổng chi tiêu.
73 net_cash_flow = total_income - total_expenses
74 # Tính tỷ lệ tiêu dùng, cho biết chi tiêu chiếm bao nhiêu phần trăm thu nhập.
75 consumption_ratio = total_expenses / total_income * 100
76
77 # Tạo bảng kết quả tóm tắt để lưu các chỉ số chính ra CSV.
78 pd.DataFrame({
79     "metric": ["Tổng thu nhập", "Tổng chi tiêu", "Dòng tiền ròng", "Tỷ lệ tiêu dùng so với thu nhập (%)"],
80     "value": [total_income, total_expenses, net_cash_flow, consumption_ratio]
81 }).to_csv(out / "01_can_can_thanh_khoan.csv", index=False, encoding="utf-8-sig")
82
83 # Tạo khung biểu đồ với kích thước 9 x 5.5 inch.
84 plt.figure(figsize=(9, 5.5))
85 # Khai báo nhãn cho 3 cột trên biểu đồ.
86 labels = ["Tổng thu nhập", "Tổng chi tiêu", "Dòng tiền ròng"]
87 # Khai báo giá trị tương ứng với từng cột.
88 values = [total_income, total_expenses, net_cash_flow]
89 # Gán màu: xanh lá cho thu nhập, đỏ cho chi tiêu, xanh dương cho dòng tiền ròng.
90 colors = ["#3ad13a", "#e73737", "#3283be"]
91 # Vẽ biểu đồ cột từ labels và values.
92 bars = plt.bar(labels, values, color=colors)
93 # Đặt tiêu đề biểu đồ mục 1 và in đậm.
94 plt.title("CÁN CÂN THANH KHOẢN TIỀN MẶT NGẮN HẠN", fontweight="bold")
95 # Đặt tên trục Y là Số tiền.
96 plt.ylabel("Số tiền")
97 # Duyệt qua từng cột để ghi giá trị lên đầu cột.
98 for bar, value in zip(bars, values):
99     plt.text(bar.get_x() + bar.get_width() / 2, bar.get_height(), f"{value:,.0f}",
100             ha="center", va="bottom", fontsize=10, fontweight="bold")
101 plt.text(0.5, max(values) * 0.82, f"Tỷ lệ tiêu dùng: {consumption_ratio:.2f}%",
102         ha="center", bbox=dict(boxstyle="round", facecolor="white", alpha=0.9))
103 plt.legend(handles=[
104     Patch(color="#3ad13a", label="Thu nhập"),
105     Patch(color="#e73737", label="Chi tiêu"),
106     Patch(color="#3283be", label="Dòng tiền ròng"),
107 ], loc="upper right")
108 save_chart("01_column_chart_can_can_thanh_khoan.png")
```



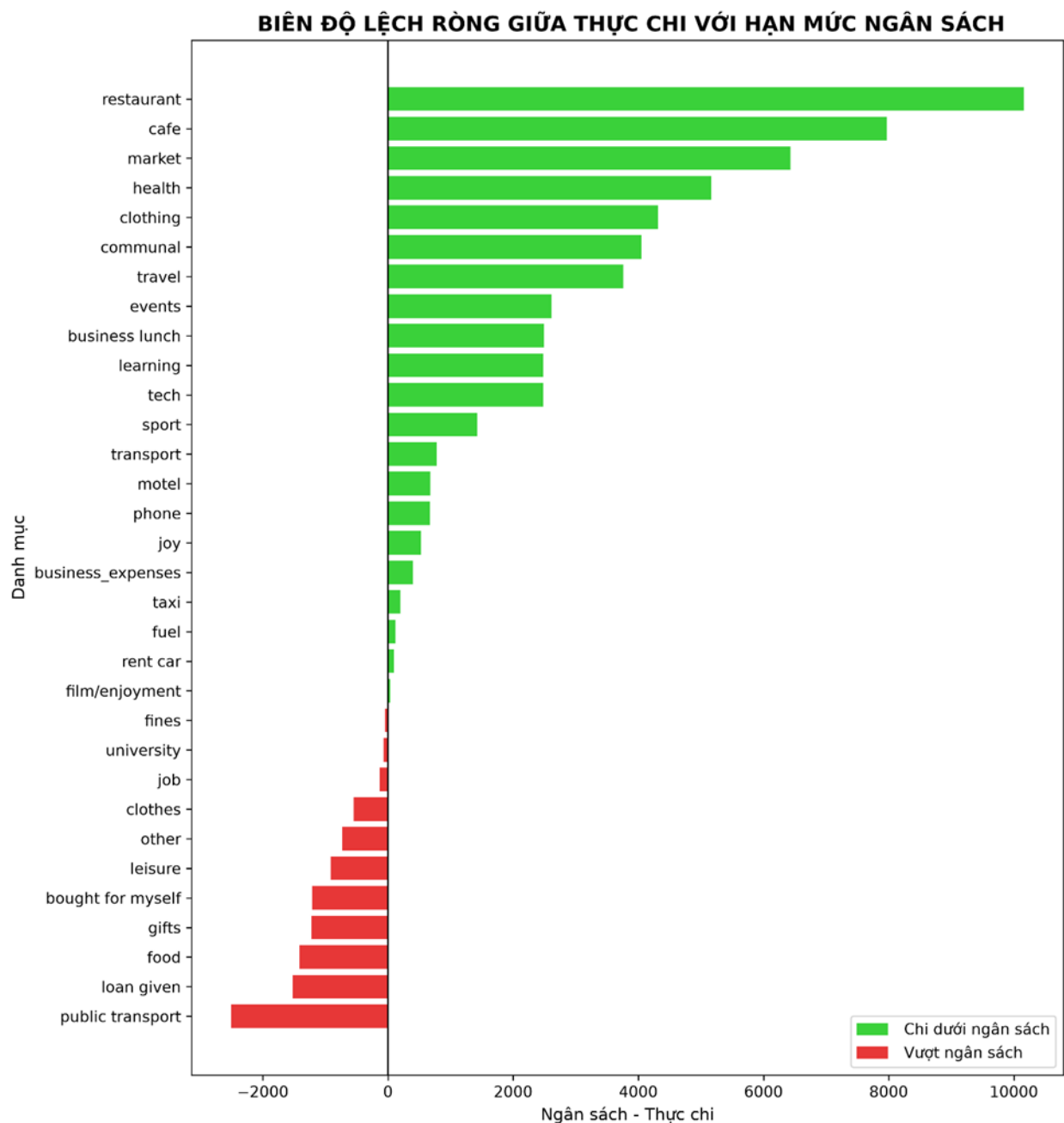
Hình 3.1: Cân cân thanh khoản tiền mặt ngắn hạn

NHẬN XÉT

- Biểu đồ cho thấy tổng thu nhập cao hơn tổng chi tiêu, chứng tỏ dòng tiền của người dùng vẫn đang ở trạng thái dương. Điều này phản ánh người dùng chưa rơi vào tình trạng thiếu hụt tiền mặt trong ngắn hạn.
- Đặc biệt, tỷ lệ tiêu dùng ở mức 54.45% khẳng định tập mẫu khách hàng nhìn chung đang sở hữu một "tấm đệm thanh khoản" tương đối an toàn. Với lượng dòng tiền ròng thặng dư đạt 11,840.00, khả năng kháng cự trước các rủi ro sụt giảm thu nhập đột ngột của nhóm này được đánh giá ở mức cao. Đây chính là chân dung đối tượng mục tiêu lý tưởng cho các dòng sản phẩm đầu tư, tích lũy dài hạn hoặc bảo hiểm rủi ro từ các tổ chức tài chính.
- Tuy nhiên, dưới góc độ tối ưu hóa tài chính cá nhân, việc tỷ lệ chi tiêu vẫn chiếm hơn một nửa thu nhập cho thấy không gian tích lũy vẫn còn dư địa để cải thiện. Nếu có các giải pháp thắt chặt kỷ luật ngân sách tốt hơn, năng lực tạo quỹ dự phòng dài hạn của tập khách hàng này sẽ còn đạt hiệu suất ấn tượng hơn nữa

3.2. BIÊN ĐỘ LỆCH RÒNG GIỮA THỰC CHI VỚI HẠN MỨC NGÂN SÁCH

```
114 # 2. BIÊN ĐỘ LỆCH RÒNG GIỮA THỰC CHI VỚI HẠN MỨC NGÂN SÁCH
115 # Chuẩn hóa tên danh mục trong bảng chỉ tiêu để gộp dữ liệu chính xác hơn.
116 expenses["category_norm"] = expenses["category"].map(normalize_category)
117 # Chuẩn hóa tên danh mục trong bảng ngân sách.
118 budget["category_norm"] = budget["category"].map(normalize_category)
119 # Tính tổng chỉ tiêu thực tế theo từng danh mục.
120 actual_spending = expenses.groupby("category_norm", as_index=False)["amount"].sum().rename(columns={"amount": "actual_spending"})
121 # Tính tổng ngân sách đặt ra theo từng danh mục.
122 budget_amount = budget.groupby("category_norm", as_index=False)["amount"].sum().rename(columns={"amount": "budget_amount"})
123 # Gộp ngân sách và thực chi theo danh mục; thay giá trị thiếu bằng 0.
124 budget_compare = budget_amount.merge(actual_spending, on="category_norm", how="outer").fillna(0)
125 # Tính độ lệch ngân sách = ngân sách - thực chi.
126 budget_compare["variance"] = budget_compare["budget_amount"] - budget_compare["actual_spending"]
127 # Gán nhãn vượt ngân sách nếu variance âm, chi dưới ngân sách nếu variance dương.
128 budget_compare["status"] = np.where(budget_compare["variance"] < 0, "Vượt ngân sách", "Chi dưới ngân sách")
129 # Sắp xếp theo độ lệch để biểu đồ diverging bar dễ đọc.
130 budget_compare = budget_compare.sort_values("variance")
131 budget_compare.to_csv(out / "02_do_lech_ngan_sach.csv", index=False, encoding="utf-8-sig")
132
133 plt.figure(figsize=(10, max(7, len(budget_compare) * 0.33)))
134 plt.barh(budget_compare["category_norm"], budget_compare["variance"],
135          color=np.where(budget_compare["variance"] >= 0, "#3ad13a", "#e73737"))
136 plt.axvline(0, color="black", linewidth=1)
137 plt.title(" BIÊN ĐỘ LỆCH RÒNG GIỮA THỰC CHI VỚI HẠN MỨC NGÂN SÁCH", fontweight="bold")
138 plt.xlabel("Ngân sách - Thực chi")
139 plt.ylabel("Danh mục")
140 plt.legend(handles=[
141     Patch(color="#3ad13a", label=" Chi dưới ngân sách"),
142     Patch(color="#e73737", label=" Vượt ngân sách"),
143 ], loc="lower right")
144 save_chart("02_diverging_bar_chart_ngan_sach.png")
```



Hình 3.2: Biên độ lệch ròng giữa thực chi với hạn mức ngân sách

NHẬN XÉT

- Biểu đồ bộc lộ hai xu hướng hành vi đối lập rõ rệt trong việc thực thi kỷ luật ngân sách của người dùng. Một mặt, người dùng thể hiện sự thất lưng buộc bụng cực kỳ ấn tượng ở các danh mục dịch vụ lớn, tạo ra thặng dư ngân sách khổng lồ tại mục nhà hàng (restaurant dư +10,160.79) và cafe dư +7,968.40.

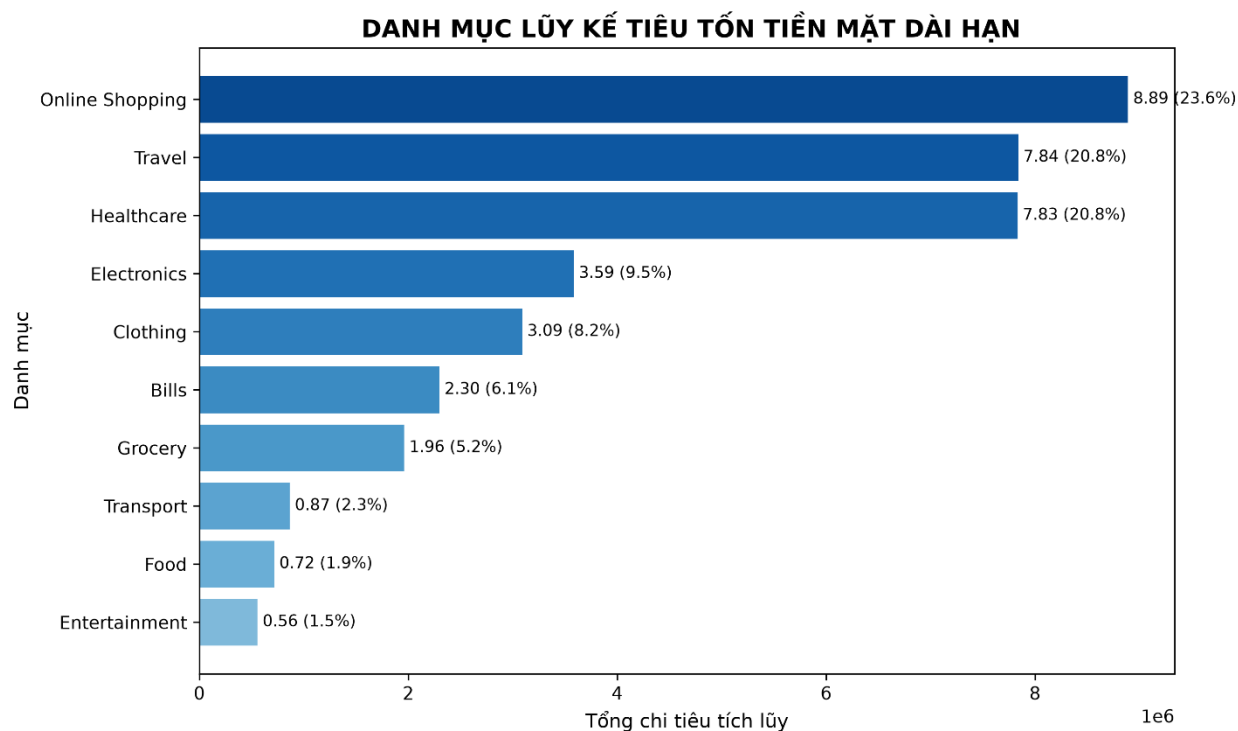
- Tuy nhiên, toàn bộ thành quả tích lũy trên đã bị triệt tiêu phần lớn bởi các khoản chi phát sinh ngẫu hứng, hoàn toàn nằm ngoài kế hoạch ban đầu (ngân sách bằng 0). Điển hình là chi phí giao thông công cộng (public transport thâm hụt -2,505.00), cho vay cá nhân (loan given thâm hụt -1,523.00), và mua sắm ngẫu hứng (bought for myself thâm hụt -1,213.00). Điều này phản ánh tư duy quản lý tài chính theo kiểu "thắt chặt cửa lớn, lỏng lẻo cửa sau" – kiểm soát tốt các khoản chi xa xỉ nhưng lại để dòng tiền rò rỉ ở các khoản chi nhỏ phát sinh hàng ngày.

3.3. DANH MỤC LŨY KẾ TIÊU TỐN TIỀN MẶT DÀI HẠN

```

150 # 3. DANH MỤC LŨY KẾ TIÊU TỐN TIỀN MẶT DÀI HẠN
151 # Tạo bản sao dữ liệu extended để xử lý riêng mà không làm thay đổi dữ liệu gốc.
152 extended_expense = extended.copy()
153 # Kiểm tra dữ liệu có cột loại giao dịch hay không.
154 if "TransactionType" in extended_expense.columns:
155     temp = extended_expense[
156         extended_expense["TransactionType"].astype(str).str.lower().str.contains("expense|debit|spent|payment", na=False)
157     ]
158     if not temp.empty:
159         extended_expense = temp
160
161 # Tính tổng số tiền theo từng danh mục và sắp xếp giảm dần.
162 long_term = extended_expense.groupby("Category", as_index=False)["Amount"].sum().sort_values("Amount", ascending=False)
163 # Tính tỷ trọng phần trăm của từng danh mục trên tổng chi tiêu.
164 long_term["share_percent"] = long_term["Amount"] / long_term["Amount"].sum() * 100
165 long_term.to_csv(out / "03_tich_luy_dai_han_theo_danh_muc.csv", index=False, encoding="utf-8-sig")
166
167 # Lấy 10 danh mục có tổng chi tiêu cao nhất để vẽ biểu đồ.
168 top_long_term = long_term.head(10)
169 gradient_colors = [plt.cm.Blues(0.9 - i * (0.45 / max(len(top_long_term) - 1, 1))) for i in range(len(top_long_term))]
170 plt.figure(figsize=(10, 6))
171 bars = plt.barh(top_long_term["Category"], top_long_term["Amount"], color=gradient_colors)
172 plt.gca().invert_yaxis()
173 plt.title("DANH MỤC LŨY KẾ TIÊU TỐN TIỀN MẶT DÀI HẠN", fontweight="bold")
174 plt.xlabel("Tổng chi tiêu tích lũy")
175 plt.ylabel("Danh mục")
176 for bar, value, percent in zip(bars, top_long_term["Amount"], top_long_term["share_percent"]):
177     plt.text(value, bar.get_y() + bar.get_height() / 2, f"{value / 1e6:.2f} ({percent:.1f}%)",
178             va="center", fontsize=9)
179 save_chart("03_bar_chart_tieu_dung_tich_luy.png")

```



Hình 3.3: Danh mục lũy kế tiêu tốn tiền mặt dài hạn

NHẬN XÉT

- Phân tích cấu trúc dòng tiền dài hạn cho thấy chi phí bị thất nút cổ chai vào ba nhóm cấu phần lớn nhất là Mua sắm trực tuyến (Online Shopping chiếm 23.62%), Du lịch (Travel chiếm 20.83%), và Y tế (Healthcare chiếm 20.81%). Tổng tỷ trọng của "bộ ba" này chiếm tới hơn 65% tổng chi tiêu xã hội
- Việc chi phí y tế chiếm tỷ trọng lớn báo hiệu nhu cầu phòng ngừa rủi ro sức khỏe bằng các giải pháp bảo hiểm rủi ro là rất hiện hữu. Tuy nhiên, việc mua sắm online và du lịch ngốn gần một nửa quỹ tiền dài hạn phản ánh lối sống ưu tiên trải nghiệm tức thời và tiêu dùng nhanh của thế hệ mới. Đây là rào cản hành vi lớn nhất làm suy giảm năng lực tích lũy để đầu tư vào các tài sản có giá trị lớn hoặc tạo lập quỹ hưu trí vững chắc trong tương lai.

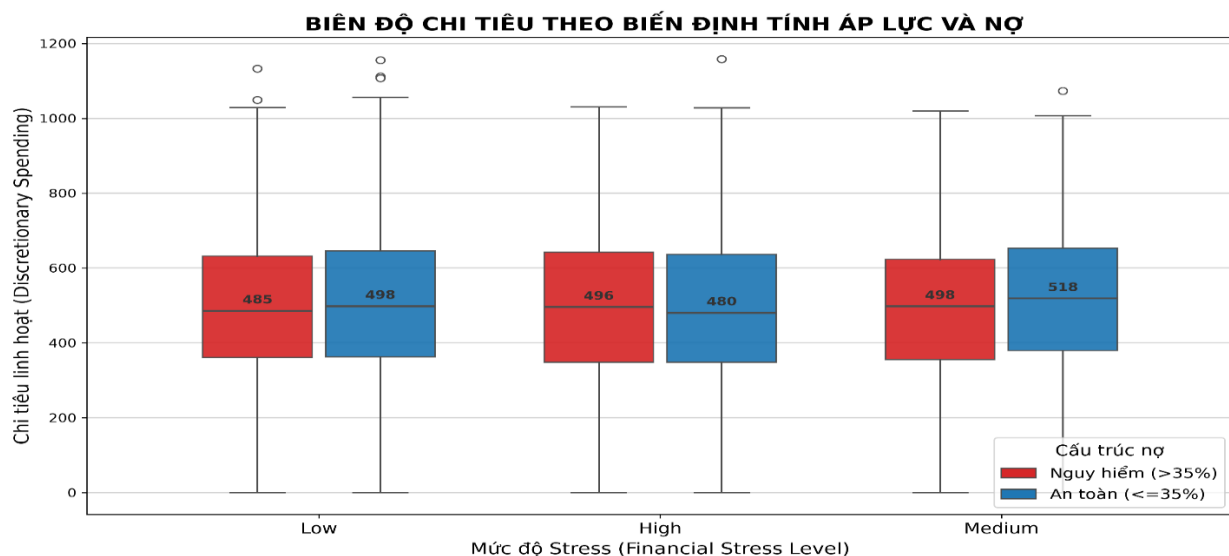
3.4. BIÊN ĐỘ CHI TIÊU THEO BIẾN ĐỊNH TÍNH ÁP LỰC VÀ NỢ

```
185 # 4. BIÊN ĐỘ CHI TIÊU THEO BIẾN ĐỊNH TÍNH ÁP LỰC VÀ NỢ
186 # Lấy 3 cột cần thiết cho biểu đồ boxplot: stress, tỷ lệ nợ/thu nhập và chi tiêu linh hoạt.
187 df4 = tracker[["financial_stress_level", "debt_to_income_ratio", "discretionary_spending"]].copy()
188 # Loại bỏ giá trị vô cực và giá trị thiếu để boxplot không bị lỗi.
189 df4 = df4.replace([np.inf, -np.inf], np.nan).dropna()
190 # Chuẩn hóa tên mức độ stress về dạng Low, Medium, High.
191 df4["financial_stress_level"] = df4["financial_stress_level"].astype(str).str.strip().str.capitalize()
192
193 # Thiết lập thứ tự mức stress theo mẫu báo cáo.
194 stress_order = ["Low", "High", "Medium"]
195 df4 = df4[df4["financial_stress_level"].isin(stress_order)].copy()
196 # Đặt ngưỡng DTI là 35% để phân loại nợ an toàn/nguy hiểm.
197 debt_threshold = 0.35
198 # Phân loại cấu trúc nợ: DTI > 35% là nguy hiểm, ngược lại là an toàn.
199 df4["debt_structure"] = np.where(df4["debt_to_income_ratio"] > debt_threshold, "Nguy hiểm (>35%)", "An toàn (<=35%)")
200
201 # Tạo bảng thống kê theo từng cặp mức stress và cấu trúc nợ.
202 summary_4 = df4.groupby(["financial_stress_level", "debt_structure"]).agg(
203     so_luong=("discretionary_spending", "count"),
204     low_q1=("discretionary_spending", lambda x: x.quantile(0.25)),
205     trung_binh=("discretionary_spending", "mean"),
206     trung_vi=("discretionary_spending", "median"),
207     high_q3=("discretionary_spending", lambda x: x.quantile(0.75)),
208     min_value=("discretionary_spending", "min"),
209     max_value=("discretionary_spending", "max"),
210 ).reset_index()
211 summary_4.to_csv(out / "04_so_sanh_nhom_rui_ro.csv", index=False, encoding="utf-8-sig")
212
213 red = "#d62728"
214 blue = "#1f77b4"
215 # Tạo các danh sách để lưu vị trí, dữ liệu và màu sắc cho boxplot.
216 positions, box_data, box_colors = [], [], []
217 # offset giúp hai boxplot trong cùng một mức stress nằm cạnh nhau.
218 offset = 0.18
219 # Duyệt từng mức stress để lấy dữ liệu nhóm nguy hiểm và nhóm an toàn.
220 for i, stress in enumerate(stress_order, start=1):
221     dangerous_data = df4[(df4["financial_stress_level"] == stress) & (df4["debt_structure"] == "Nguy hiểm (>35%))"]["discretionary_spending"].dropna()
222     safe_data = df4[(df4["financial_stress_level"] == stress) & (df4["debt_structure"] == "An toàn (<=35%))"]["discretionary_spending"].dropna()
223     box_data.extend([dangerous_data, safe_data])
224     positions.extend([i - offset, i + offset])
225     box_colors.extend([red, blue])
```

```

227 # Tạo khung biểu đồ lớn hơn để hiển thị 6 boxplot rõ ràng.
228 plt.figure(figsize=(12, 7))
229 # Vẽ boxplot cho các nhóm chỉ tiêu linh hoạt.
230 box = plt.boxplot(
231     box_data, positions=positions, widths=0.32, patch_artist=True, showfliers=True,
232     medianprops=dict(color="#4d4d4d", linewidth=1.7),
233     whiskerprops=dict(color="#4d4d4d", linewidth=1.2),
234     capprops=dict(color="#4d4d4d", linewidth=1.2),
235     flierprops=dict(marker="o", markerfacecolor="white", markeredgcolor="#4d4d4d", markersize=6)
236 )
237 # Gán màu đỏ/xanh cho từng boxplot theo nhóm nợ.
238 for patch, color in zip(box["boxes"], box_colors):
239     patch.set_facecolor(color)
240     patch.set_alpha(0.92)
241     patch.set_edgcolor("#4d4d4d")
242 # Duyệt từng đường trung vị để ghi giá trị median lên biểu đồ.
243 for i, median in enumerate(box["medians"]):
244     x_coords = median.get_xdata()
245     y_coords = median.get_ydata()
246     x_pos = x_coords.mean()
247     y_pos = y_coords[0]
248     plt.text(
249         x_pos,
250         y_pos + 12,
251         f"{int(y_pos)}",
252         ha="center",
253         va="bottom",
254         fontsize=10,
255         fontweight="bold",
256         color="#333333"
257     )
258 plt.title("BIÊN ĐỘ CHI TIÊU THEO BIẾN ĐỊNH TÍNH ÁP LỰC VÀ NỢ", fontsize=16, fontweight="bold")
259 plt.xlabel("Mức độ Stress (Financial Stress Level)", fontsize=14)
260 plt.ylabel("Chi tiêu linh hoạt (Discretionary Spending)", fontsize=14)
261 plt.xticks([1, 2, 3], stress_order, fontsize=13)
262 plt.grid(axis="y", color="#cccccc", linewidth=1, alpha=0.9)
263 plt.legend(handles=[
264     Patch(facecolor=red, edgcolor="#4d4d4d", label="Nguy hiểm (>35%)",
265     Patch(facecolor=blue, edgcolor="#4d4d4d", label="An toàn (<=35%)",
266 ], title="Cấu trúc nợ", loc="lower right", frameon=True, fontsize=12, title_fontsize=13)
267 save_chart("04_boxplot_no_stress.png")

```



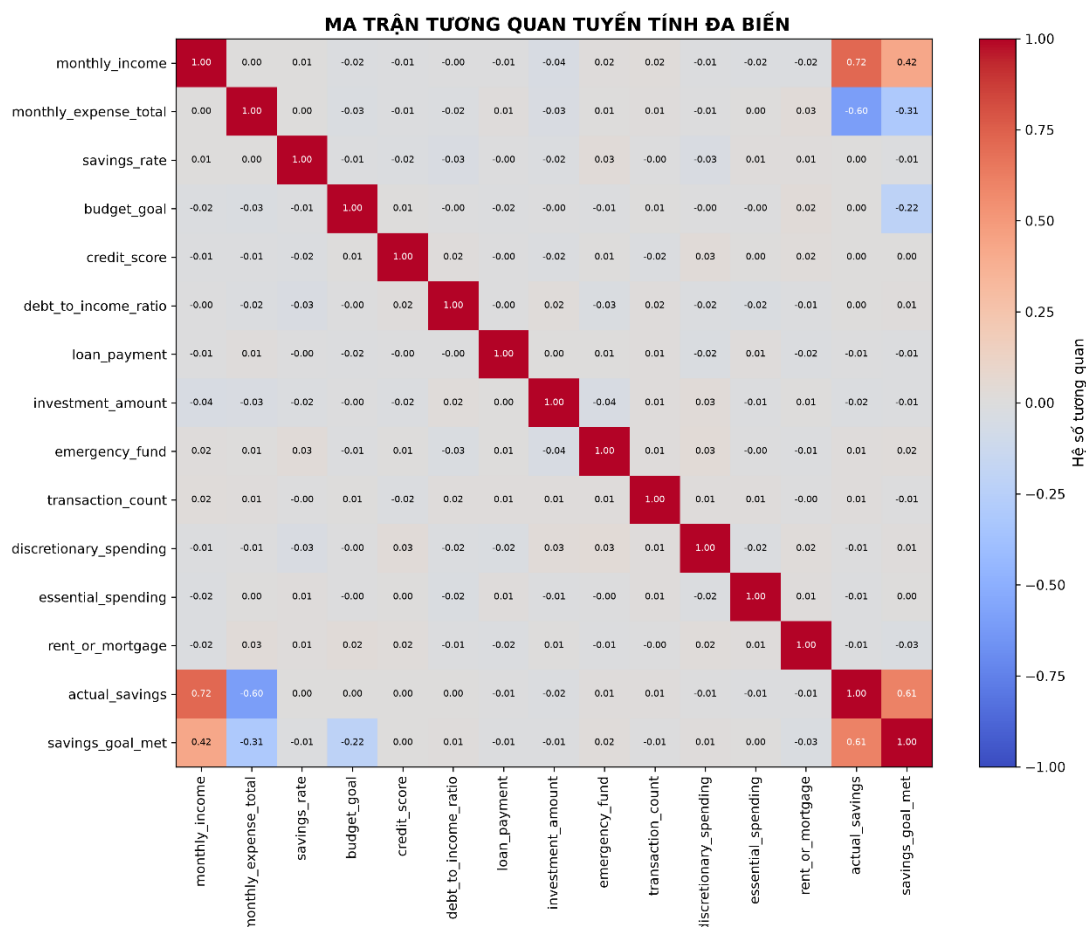
Hình 3.4: Biên độ chi tiêu theo biến định tính áp lực và nợ

NHẬN XÉT

- Phân khúc khách hàng sở hữu mức tích lũy trung vị tối ưu nhất hệ thống lại rơi vào nhóm có áp lực tài chính Trung bình (Medium Stress) kết hợp cấu trúc nợ an toàn, đạt mức 518.99. Chỉ số này vượt trội hơn cả nhóm có áp lực tài chính thấp (Low Stress chỉ đạt mức trung vị từ 485.97 đến 498.34).
- Điều này chứng minh một mức độ áp lực vừa phải (Medium Stress) đóng vai trò như một động lực tâm lý tích cực, thúc đẩy cá nhân thắt chặt kỷ luật tiền tệ để tích lũy nhiều hơn. Tuy nhiên, một khi áp lực tài chính vượt ngưỡng chuyển sang trạng thái Cao (High Stress), năng lực kiểm soát dòng tiền lập tức bị suy nhược, đẩy số dư tích lũy trung vị rơi xuống mức đáy toàn hệ thống (480.76). Do đó, các tổ chức tín dụng cần đặc biệt thận trọng khi thiết kế các gói vay, tránh đẩy khách hàng từ vùng áp lực lành mạnh sang vùng kiệt quệ tài chính.

3.5. MA TRẬN TƯƠNG QUAN TUYẾN TÍNH ĐA BIẾN

```
273 # 5. MA TRẬN TƯƠNG QUAN TUYẾN TÍNH ĐA BIẾN
274 # Danh sách các biến tài chính dùng để tính hệ số tương quan.
275 corr_columns = [
276     "monthly_income", "monthly_expense_total", "savings_rate", "budget_goal", "credit_score",
277     "debt_to_income_ratio", "loan_payment", "investment_amount", "emergency_fund",
278     "transaction_count", "discretionary_spending", "essential_spending",
279     "rent_or_mortgage", "actual_savings", "savings_goal_met",
280 ]
281 # Tính ma trận tương quan Pearson giữa các biến số.
282 corr = tracker[corr_columns].corr(numeric_only=True)
283 corr.to_csv(out / "05_ma_tran-tuong-quan.csv", encoding="utf-8-sig")
284
285 plt.figure(figsize=(12, 10))
286 # Vẽ heatmap, màu đỏ thể hiện tương quan dương và màu xanh thể hiện tương quan âm.
287 image = plt.imshow(corr, cmap="coolwarm", vmin=-1, vmax=1, aspect="auto")
288 plt.colorbar(image, label="Hệ số tương quan")
289 plt.xticks(range(len(corr.columns)), corr.columns, rotation=90)
290 plt.yticks(range(len(corr.index)), corr.index)
291 plt.title(" MA TRẬN TƯƠNG QUAN TUYẾN TÍNH ĐA BIẾN", fontweight="bold")
292 # Duyệt từng ô trong ma trận để ghi hệ số tương quan lên heatmap.
293 for i in range(corr.shape[0]):
294     for j in range(corr.shape[1]):
295         value = corr.iloc[i, j]
296         plt.text(j, i, f"{value:.2f}", ha="center", va="center", fontsize=7,
297                 color="white" if abs(value) > 0.55 else "black")
298 save_chart("05_heatmap-tuong-quan.png")
```



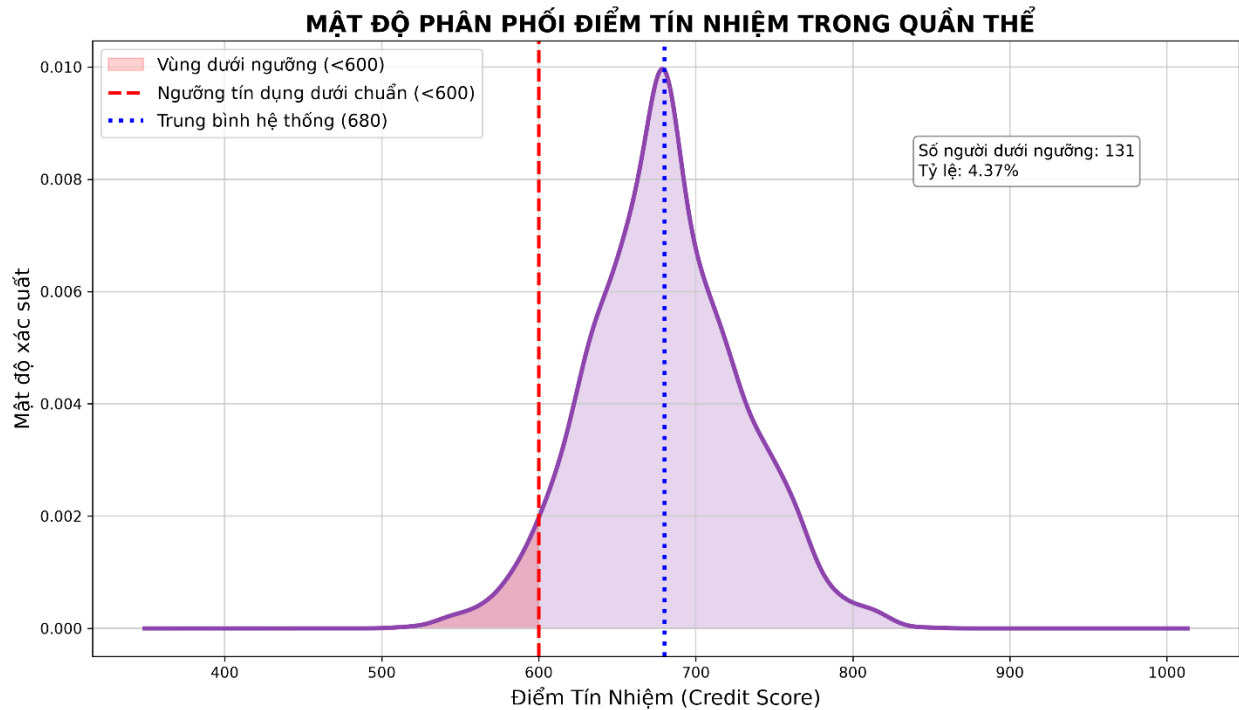
Hình 3.5: Ma trận tương quan tuyến tính đa biến

NHẬN XÉT

- Ma trận Pearson cung cấp một cái nhìn định lượng chuẩn xác về mối quan hệ nhân quả giữa các biến số tài chính.
- Kết quả cho thấy số dư tích lũy thực tế (actual_savings) chịu tác động cực kỳ mạnh mẽ bởi hai yếu tố đối lập: tỷ lệ thuận với thu nhập hàng tháng (monthly_income đạt +0.716) và tỷ lệ nghịch sâu sắc với tổng chi tiêu hàng tháng (monthly_expense_total đạt -0.600). Đáng chú ý, hệ số tương quan chéo giữa các biến độc lập khác (ví dụ như giữa điểm tín dụng credit_score và tỷ lệ nợ trên thu nhập debt_to_income_ratio chỉ ở mức +0.015) hầu như bằng 0.

3.6. MẬT ĐỘ PHÂN PHỐI ĐIỂM TÍN NHIỆM TRONG QUẦN THỂ

```
304 # 6. MẬT ĐỘ PHÂN PHỐI ĐIỂM TÍN NHIỆM TRONG QUẦN THỂ
305 # Chuyển credit_score sang số và loại bỏ giá trị thiếu/vô cực.
306 credit = pd.to_numeric(tracker["credit_score"], errors="coerce").replace([np.inf, -np.inf], np.nan).dropna()
307 # Đặt ngưỡng tín dụng dưới chuẩn là 600 điểm.
308 subprime_threshold = 600
309 # Tính điểm tín dụng trung bình của toàn bộ hệ thống.
310 system_mean = credit.mean()
311 # Đếm số người có điểm tín dụng dưới ngưỡng 600.
312 below_count = int((credit < subprime_threshold).sum())
313 # Tính tỷ lệ phần trăm người dưới ngưỡng.
314 below_rate = below_count / len(credit) * 100
315
316 # Tạo bảng kết quả tóm tắt để lưu các chỉ số chính ra CSV.
317 pd.DataFrame({
318     "metric": ["Điểm tín dụng trung bình hệ thống", "Ngưỡng tín dụng dưới chuẩn", "Số người dưới ngưỡng", "Tỷ lệ dưới ngưỡng (%)"],
319     "value": [system_mean, subprime_threshold, below_count, below_rate]
320 }).to_csv(out / "06_phan_phoi_diem_tin_nhiem.csv", index=False, encoding="utf-8-sig")
321
322 # Tạo biểu đồ tạm để lấy dữ liệu tọa độ KDE từ pandas.
323 temporary_figure, temporary_axis = plt.subplots()
324 # Vẽ KDE tạm để pandas tính đường mật độ xác suất.
325 credit.plot(kind="kde", ax=temporary_axis)
326 line = temporary_axis.lines[0]
327 # Lấy tọa độ X của đường KDE.
328 x_data = line.get_xdata()
329 # Lấy tọa độ Y của đường KDE.
330 y_data = line.get_ydata()
331 plt.close(temporary_figure)
332
333 # Tạo khung biểu đồ lớn hơn để hiển thị 6 boxplot rõ ràng.
334 plt.figure(figsize=(12, 7))
335 plt.plot(x_data, y_data, color="#8e44ad", linewidth=3)
336 plt.fill_between(x_data, y_data, color="#8e44ad", alpha=0.22)
337 # Tạo mask để xác định phần đường KDE nằm dưới ngưỡng 600.
338 below_mask = x_data <= subprime_threshold
339 plt.fill_between(x_data[below_mask], y_data[below_mask], color="red", alpha=0.18, label="Vùng dưới ngưỡng (<600)")
340 plt.axvline(subprime_threshold, color="red", linestyle="--", linewidth=2.4, label=f"Ngưỡng tín dụng dưới chuẩn (<{subprime_threshold})")
341 plt.axvline(system_mean, color="blue", linestyle=":", linewidth=2.8, label=f"Trung bình hệ thống ({system_mean:.0f})")
342 plt.title("MẬT ĐỘ PHÂN PHỐI ĐIỂM TÍN NHIỆM TRONG QUẦN THỂ", fontsize=16, fontweight="bold")
343 plt.xlabel("Điểm Tín Nhiệm (Credit Score)", fontsize=14)
344 plt.ylabel("Mật độ xác suất", fontsize=14)
345 plt.grid(color="#cccccc", linewidth=1, alpha=0.9)
346 plt.legend(loc="upper left", frameon=True, fontsize=12)
347
348 plt.text(0.72, 0.78, f"Số người dưới ngưỡng: {below_count}\nTỷ lệ: {below_rate:.2f}%",
349         transform=plt.gca().transAxes, fontsize=11,
350         bbox=dict(boxstyle="round", facecolor="white", edgecolor="gray", alpha=0.85))
351 save_chart("06_kde_credit_score.png")
```



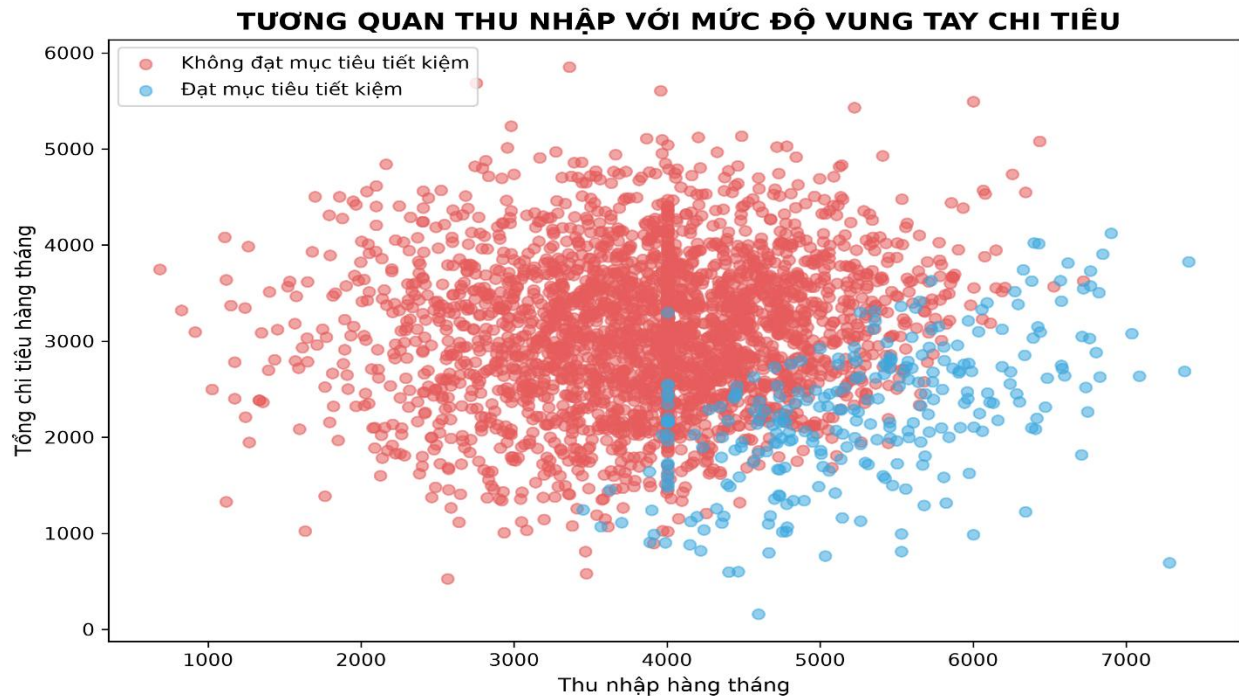
Hình 3.6: Mật độ phân phối điểm tín nhiệm trong quản thể

NHẬN XÉT

- Điểm tín dụng trung bình toàn hệ thống neo ở mức 680.08 điểm, phản ánh chất lượng tín dụng chung của tập mẫu nằm trong phân hạng Khá và tương đối an toàn.
- uy nhiên, hệ thống ghi nhận 131 cá nhân lọt vào vùng rủi ro dưới ngưỡng chuẩn 600 điểm, tương đương với tỷ lệ nợ xấu tiềm ẩn là 4.37%. Mặc dù tỷ lệ này nằm trong tầm kiểm soát, việc xuất hiện nhóm dưới chuẩn buộc các nhà quản trị rủi ro phải áp dụng quy tắc cứng (Hard Cut-off Rule) tại mốc 600 điểm.
- Bất kỳ hành vi nói lỏng chính sách cho vay nào đối với nhóm này đều có thể kích hoạt rủi ro nợ xấu dây chuyền, đe dọa trực tiếp đến tính thanh khoản của hệ thống.

3.7. TƯƠNG QUAN THU NHẬP VỚI MỨC ĐỘ VUNG TAY CHI TIÊU

```
356 # 7. TƯƠNG QUAN THU NHẬP VỚI MỨC ĐỘ VUNG TAY CHI TIÊU
357 # Tạo bảng thống kê theo nhóm đạt/không đạt mục tiêu tiết kiệm.
358 summary_7 = tracker.groupby("savings_goal_met", as_index=False).agg(
359     count=("user_id", "count"),
360     avg_income=("monthly_income", "mean"),
361     avg_expense=("monthly_expense_total", "mean"),
362     avg_savings_rate=("savings_rate", "mean"),
363 )
364 summary_7["status"] = summary_7["savings_goal_met"].map({0: "Không đạt mục tiêu tiết kiệm", 1: "Đạt mục tiêu tiết kiệm"})
365 summary_7.to_csv(out / "07_thu_nhap_va_bay_lam_phat_loi_song.csv", index=False, encoding="utf-8-sig")
366
367 # Lấy dữ liệu thu nhập, chi tiêu và trạng thái tiết kiệm để vẽ scatter plot.
368 scatter_df = tracker[["monthly_income", "monthly_expense_total", "savings_goal_met"]].replace([np.inf, -np.inf], np.nan).dropna()
369 # Gán thu nhập hàng tháng làm biến X.
370 x = scatter_df["monthly_income"].astype(float)
371 # Gán tổng chi tiêu hàng tháng làm biến Y.
372 y = scatter_df["monthly_expense_total"].astype(float)
373 # Tính hệ số tương quan giữa thu nhập và chi tiêu.
374 correlation = x.corr(y)
375 # Tính hệ số hồi quy tuyến tính bậc 1.
376 regression_coef = np.polyfit(x.to_numpy(), y.to_numpy(), 1)
377 line_x = np.linspace(x.min(), x.max(), 100)
378 line_y = regression_coef[0] * line_x + regression_coef[1]
379
380 plt.figure(figsize=(9, 6))
381 for value, color, label in [
382     (0, "#e75c5c", "Không đạt mục tiêu tiết kiệm"),
383     (1, "#3ca9e0", "Đạt mục tiêu tiết kiệm"),
384 ]:
385     subset = scatter_df[scatter_df["savings_goal_met"] == value]
386     plt.scatter(subset["monthly_income"], subset["monthly_expense_total"], alpha=0.55, color=color, label=label)
387 plt.title("TƯƠNG QUAN THU NHẬP VỚI MỨC ĐỘ VUNG TAY CHI TIÊU", fontweight="bold")
388 plt.xlabel("Thu nhập hàng tháng")
389 plt.ylabel("Tổng chi tiêu hàng tháng")
390 plt.legend(loc="upper left")
391 save_chart("07_scatter_income_vs_expense.png")
```



Hình 3.7: Tương quan thu nhập với mức độ vung tay chi tiêu

NHẬN XÉT

- Biểu đồ cung cấp một bằng chứng định lượng rõ nét cho lý thuyết kinh tế về Bẫy lạm phát lỗi sống. Nhóm cán đích mục tiêu tiết kiệm thành công (Nhóm 1) có thu nhập bình quân vượt trội (5,282.38) và duy trì chi tiêu thắt lưng buộc bụng cực kỳ nghiêm túc (2,240.50).
- Nhóm thất bại (Nhóm 0) chiếm tỷ lệ áp đảo với 2.723 người (90.77%), mặc dù họ sở hữu mức thu nhập trung bình khá tốt là 3,867.72 nhưng lại để mức chi tiêu leo thang quá mạnh theo thu nhập (3,090.13). Điều đáng chú ý là tỷ lệ tiết kiệm trên lý thuyết của cả hai nhóm là tương đương nhau (22%).
- Điều này đưa ra một lời cảnh báo: Thu nhập tăng lên không đồng nghĩa với an toàn tài chính; nếu không kiểm soát được hành vi chi tiêu thực tế, chiếc bẫy lạm phát lỗi sống sẽ triệt tiêu hoàn toàn năng lực tích lũy số tuyệt đối của người dùng.

3.8. ĐỀ XUẤT GIẢI PHÁP

Từ kết quả phân tích hệ thống biểu đồ, chúng tôi đề xuất một số giải pháp chiến lược như sau:

- Thứ nhất, xây dựng kế hoạch chi tiêu theo tỷ lệ thu nhập: Chia thu nhập thành các quỹ cố định (như chi tiêu thiết yếu, tiết kiệm, đầu tư). Việc duy trì tỷ lệ chi tiêu thực tế ở mức 54.45% cho thấy người dùng có nền tảng thặng dư rất tốt (11,840.00), cần tự động hóa việc trích lập các quỹ này ngay đầu tháng để hạn chế tình trạng chi tiêu ngẫu hứng.
- Thứ hai, thiết lập ngân sách cụ thể cho từng danh mục: Đối với các danh mục thường xuyên vượt ngân sách hoặc phát sinh ngẫu hứng (như public transport thâm hụt -2,505.00 hay loan given thâm hụt -1,523.00), người dùng nên cài đặt cảnh báo tự động khi mức chi đạt 80% hạn mức để kịp thời điều chỉnh hành vi.

- Thứ ba, ưu tiên kiểm soát các danh mục chiếm tỷ trọng lớn trong dài hạn: Tập trung tối ưu hóa "bộ ba" đang ngốn hơn 65% tổng chi phí dài hạn bao gồm: Mua sắm trực tuyến (23.62%), Du lịch (20.83%) và Y tế (20.81%). Việc thắt chặt kỷ luật ở các nhóm này sẽ cải thiện rõ rệt năng lực tích lũy tài sản thực tế.
- Thứ tư, duy trì tỷ lệ nợ trên thu nhập (DTI) ở mức an toàn: Kiểm soát nghiêm ngặt tỷ lệ nợ không vượt quá 35%. Số liệu chứng minh nhóm giữ cấu trúc nợ an toàn và áp lực vừa phải (Medium Stress) có mức tích lũy trung vị cao nhất hệ thống (518.99); do đó, việc giảm nợ lãi suất cao sẽ giúp giải phóng áp lực và tối ưu hóa số dư tích lũy.
- Thứ năm, theo dõi điểm tín nhiệm định kỳ và quản trị rủi ro: Mặc dù điểm trung bình hệ thống đạt mức khá (680.08 điểm), vẫn có 4.37% người dùng lọt vào vùng rủi ro dưới ngưỡng 600 điểm. Nhóm này cần được áp dụng quy tắc cứng (Hard Cut-off), tập trung cải thiện lịch sử thanh toán để tránh gây xói mòn tính thanh khoản của mạng lưới.
- Thứ sáu, triệt tiêu bẫy lạm phát lối sống: Người dùng cần tránh tình trạng chi tiêu leo thang theo thu nhập. Bài học từ 2,723 cá nhân thất bại (Nhóm 0) cho thấy dù thu nhập khá tốt (3,867.72) nhưng do đề chi tiêu tăng quá mạnh (3,090.13) nên đã không đạt mục tiêu. Khi thu nhập tăng, bắt buộc phải ưu tiên tăng tỷ lệ tích lũy số tuyệt đối thay vì mở rộng nhu cầu cá nhân.

CHƯƠNG 4: PHÂN TÍCH DỮ LIỆU ĐƯA VÀO MÔ HÌNH MÁY HỌC

4.1. TIỀN XỬ LÝ VÀ MÃ HÓA

Tiền xử lý dữ liệu là bước đầu tiên và cũng là một trong những bước quan trọng nhất trong quy trình xây dựng mô hình học máy. Chất lượng dữ liệu đầu vào ảnh hưởng trực tiếp đến khả năng học của mô hình cũng như độ chính xác của kết quả dự đoán. Nếu dữ liệu chứa nhiều giá trị thiếu, dữ liệu không đồng nhất hoặc các đặc trưng không phù hợp, mô hình rất dễ học sai quy luật hoặc đưa ra kết quả dự đoán thiếu chính xác.

Trong chương trình, quá trình tiền xử lý được thực hiện theo nhiều bước nhằm đảm bảo bộ dữ liệu đạt chất lượng tốt trước khi đưa vào huấn luyện.

```
# 1. TIỀN XỬ LÝ DỮ LIỆU (DATA PREPROCESSING)
try:
    df = pd.read_csv("Track_CLEANED.csv") # Đọc file dữ liệu csv
    print("[*] Đã nạp thành công dữ liệu gốc.")
except FileNotFoundError: # Xử lý ngoại lệ nếu không tìm thấy file để tránh chương trình bị crash
    print("Lỗi: Không tìm thấy file dữ liệu Track_CLEANED.csv!")
    exit() # Dừng chương trình nếu lỗi

# Loại bỏ các dòng thiếu dữ liệu mục tiêu tiết kiệm
df = df.dropna(subset=['savings_goal_met', 'actual_savings'])

# 9 Biến đầu vào cơ sở (Base Features)
base_features = [
    "debt_to_income_ratio", "credit_score", "savings_rate",
    "emergency_fund", "loan_payment", "monthly_income",
    "monthly_expense_total", "discretionary_spending", "essential_spending"
]
# Loại lại danh sách đặc trưng: Chỉ giữ lại những cột thực sự tồn tại trong Dataframe
base_features = [col for col in base_features if col in df.columns]

# Xử lý giá trị khuyết thiếu bằng Median
for col in base_features:
    if df[col].isnull().any(): # Nếu cột có chứa giá trị NaN
        df[col] = df[col].fillna(df[col].median()) # Điền các ô trống bằng giá trị trung vị

X_base = df[base_features] # Ma trận chứa các biến đầu vào gốc

# Mục tiêu 1 cho Tầng Phân loại (0 = Không đạt, 1 = Đạt mục tiêu)
le = LabelEncoder()
y_class = le.fit_transform(df['savings_goal_met'])
target_names = le.classes_ # Lưu lại tên các nhãn gốc để dùng sau này
```

4.1.1. Đọc dữ liệu

- `df = pd.read_csv("Track_CLEANED.csv")` : Lệnh tạo ra một đối tượng **DataFrame**, đây là cấu trúc dữ liệu dạng bảng của Pandas, cho phép lưu trữ dữ liệu theo hàng và cột tương tự như bảng trong Excel hoặc cơ sở dữ liệu.
- Để tăng tính ổn định cho chương trình, quá trình đọc dữ liệu được đặt trong khối `try...except`.
- Khối lệnh `try...except` có nhiệm vụ xử lý ngoại lệ nếu tệp dữ liệu không tồn tại hoặc sai đường dẫn. Khi xảy ra lỗi, chương trình sẽ thông báo cho người dùng và kết thúc quá trình thực thi thay vì bị dừng đột ngột. Điều này giúp chương trình hoạt động ổn định hơn trong thực tế.

```
# 1. TIỀN XỬ LÝ DỮ LIỆU (DATA PREPROCESSING)
try:
    df = pd.read_csv("Track_CLEANED.csv") # Đọc file dữ liệu csv
    print("[*] Đã nạp thành công dữ liệu gốc.")
except FileNotFoundError: # Xử lý ngoại lệ nếu không tìm thấy file để tránh chương trình bị crash
    print("Lỗi: Không tìm thấy file dữ liệu Track_CLEANED.csv!")
    exit() # Dừng chương trình nếu lỗi
```

4.1.2. Loại bỏ dữ liệu không đầy đủ

- Sau khi đọc dữ liệu, chương trình tiến hành kiểm tra sự tồn tại của hai biến mục tiêu:
 - `Savings_goal_met`
 - `Actual_saving`
- Đây là hai biến đóng vai trò rất quan trọng vì:
 - **savings_goal_met** là nhãn dùng cho bài toán phân loại.
 - **actual_savings** là giá trị mục tiêu của bài toán hồi quy.
- Để đảm bảo mỗi mẫu dữ liệu đều có đầy đủ thông tin phục vụ cho cả hai bài toán, chương trình sử dụng:

```
# Loại bỏ các dòng thiếu dữ liệu mục tiêu tiết kiệm
df = df.dropna(subset=['savings_goal_met', 'actual_savings'])
```

- Hàm dropna() sẽ loại bỏ toàn bộ các dòng mà một trong hai cột trên có giá trị rỗng.
- Việc loại bỏ các bản ghi này là cần thiết bởi nếu biến mục tiêu bị thiếu thì mô hình sẽ không có dữ liệu đúng để học, dẫn đến quá trình huấn luyện không thể thực hiện hoặc kết quả học bị sai lệch.
- Khác với các cột đặc trưng đầu vào có thể thay thế giá trị thiếu, đối với biến mục tiêu thì việc loại bỏ là giải pháp phù hợp hơn nhằm đảm bảo tính chính xác của dữ liệu.

•

4.1.3. Lựa chọn đặc trưng đầu vào

- Sau khi dữ liệu được làm sạch, chương trình lựa chọn các thuộc tính được sử dụng làm đầu vào cho mô hình.

```
base_features = [
    "debt_to_income_ratio", "credit_score", "savings_rate",
    "emergency_fund", "loan_payment", "monthly_income",
    "monthly_expense_total", "discretionary_spending", "essential_spending"
]
```

- Danh sách này bao gồm **9 đặc trưng tài chính** phản ánh khả năng quản lý thu nhập và chi tiêu của người dùng. Ý nghĩa của từng biến như sau:

Biến	Ý Nghĩa
debt_to_income_ratio	Tỷ lệ nợ trên thu nhập
credit_score	Điểm tín dụng
savings_rate	Tỷ lệ tiết kiệm
emergency_fund	Quỹ dự phòng
loan_payment	Khoản trả nợ hàng tháng

monthly_income	Thu nhập hàng tháng
monthly_expense_total	Tổng chi tiêu hàng tháng
discretionary_spending	Chi tiêu không thiết yếu
essential_spending	Chi tiêu thiết yếu

- Những đặc trưng này được lựa chọn vì có ảnh hưởng trực tiếp đến khả năng tiết kiệm của mỗi cá nhân và là cơ sở để mô hình học mối quan hệ giữa tình hình tài chính và kết quả tiết kiệm.

4.1.4. Kiểm tra sự tồn tại của các cột

- Trong thực tế, bộ dữ liệu có thể thay đổi giữa các phiên bản, dẫn đến việc một số cột không còn tồn tại hoặc được đổi tên.
- Để tránh chương trình phát sinh lỗi khi truy cập các cột không tồn tại

```
# Lọc lại danh sách đặc trưng: Chỉ giữ lại những cột thực sự tồn tại trong Dataframe
base_features = [col for col in base_features if col in df.columns]
```

- Đoạn mã sẽ duyệt qua từng tên cột trong danh sách base_features và chỉ giữ lại những cột thực sự tồn tại trong DataFrame. Nhờ đó chương trình có khả năng thích ứng với nhiều phiên bản dữ liệu khác nhau mà không cần chỉnh sửa thủ công.

4.1.5. Xử lý giá trị khuyết thiếu

- Sau khi xác định được các đặc trưng đầu vào, chương trình tiếp tục kiểm tra dữ liệu bị thiếu.

```
# Xử lý giá trị khuyết thiếu bằng Median
for col in base_features:
    if df[col].isnull().any(): # Nếu cột có chứa giá trị NaN
        df[col] = df[col].fillna(df[col].median()) # Điền các ô trống bằng giá trị trung vị
```

- df[col].isnull().any() nếu cột chứa ít nhất một giá trị NaN thì chương trình sẽ thực hiện thay thế.

- `df[col] = df[col].fillna(df[col].median())` lý do là dữ liệu tài chính thường xuất hiện nhiều giá trị ngoại lệ.

Ví dụ:

3000
3200
3400
3500
50000

Nếu sử dụng Mean: Mean = 12620

Giá trị trung bình bị kéo lên rất lớn do ảnh hưởng của 50.000. Trong khi đó: Median = 3400 vẫn phản ánh đúng xu hướng của phần lớn dữ liệu.

Do đó việc sử dụng Median giúp mô hình ít bị ảnh hưởng bởi các giá trị bất thường, đồng thời duy trì được phân bố dữ liệu tự nhiên.

4.1.6. Xây dựng ma trận đặc trưng

Sau khi dữ liệu được làm sạch và hoàn thiện, chương trình tạo tập dữ liệu đầu vào:

```
X_base = df[base_features] # Ma trận chứa các biến đầu vào gốc
```

Biến `X_base` là ma trận chứa toàn bộ các đặc trưng được sử dụng để huấn luyện mô hình. Mỗi hàng biểu diễn thông tin của một khách hàng, còn mỗi cột tương ứng với một đặc trưng tài chính. Đây là tập dữ liệu đầu vào sẽ được sử dụng xuyên suốt quá trình huấn luyện các mô hình phân loại và hồi quy ở các bước tiếp theo.

4.1.7. Xây dựng biến mục tiêu

Đề tài giải quyết đồng thời hai bài toán học máy nên chương trình xây dựng hai biến mục tiêu riêng biệt. Đối với bài toán phân loại:

```
# Mục tiêu 1 cho Tầng Phân loại (0 = Không đạt, 1 = Đạt mục tiêu)
```

Biến `savings_goal_met` được mã hóa thành các giá trị số bằng **LabelEncoder**, trong đó mỗi nhãn được ánh xạ sang một số nguyên (ví dụ: *No* $\rightarrow 0$, *Yes* $\rightarrow 1$). Việc mã hóa này giúp các thuật toán học máy có thể xử lý biến mục tiêu dạng phân loại.

Đối với bài toán hồi quy:

```
# Mục tiêu 2 cho Tầng Hồi quy (Dự đoán số tiền tiết kiệm thực tế)
```

Biến `actual_savings` là giá trị số liên tục biểu diễn số tiền tiết kiệm thực tế của từng khách hàng, được sử dụng làm đầu ra cho mô hình `RandomForestRegressor`.

Như vậy, sau bước tiền xử lý, dữ liệu đã được làm sạch, lựa chọn các đặc trưng phù hợp, xử lý giá trị khuyết thiếu và xây dựng đầy đủ các biến đầu vào cũng như biến mục tiêu. Đây là nền tảng quan trọng để chuyển sang các bước chia dữ liệu, chuẩn hóa và huấn luyện mô hình học máy ở các mục tiếp theo.

4.2. MÃ HÓA DỮ LIỆU VÀ XÂY DỰNG BIẾN MỤC TIÊU

Sau khi hoàn thành bước tiền xử lý dữ liệu, chương trình tiến hành xây dựng các biến mục tiêu (Target Variables) để phục vụ cho hai bài toán học máy khác nhau. Điểm đặc biệt của đề tài là không chỉ dự đoán người dùng có đạt mục tiêu tiết kiệm hay không mà còn dự đoán số tiền tiết kiệm thực tế. Vì vậy chương trình cần xây dựng đồng thời một biến mục tiêu cho bài toán phân loại và một biến mục tiêu cho bài toán hồi quy.

4.2.1. Mã hóa biến mục tiêu cho bài toán phân loại

Trong bộ dữ liệu, cột **savings_goal_met** biểu diễn trạng thái người dùng có đạt được mục tiêu tiết kiệm hay không. Đây là dữ liệu dạng phân loại (Categorical Data), thường được biểu diễn dưới dạng chuỗi như "Yes" hoặc "No". Các thuật toán Machine Learning của Scikit-learn không thể xử lý trực tiếp dữ liệu văn bản, vì vậy cần chuyển đổi chúng sang dạng số.

Chương trình sử dụng lớp **LabelEncoder** của thư viện Scikit-learn.

```
le = LabelEncoder()
```

Dòng lệnh trên tạo ra một đối tượng LabelEncoder. Đối tượng này có nhiệm vụ ghi nhớ các nhãn xuất hiện trong dữ liệu và ánh xạ chúng sang các giá trị số nguyên.

Tiếp theo chương trình thực hiện:

```
y_class = le.fit_transform(df['savings_goal_met'])
```

Hàm `fit_transform()` bao gồm hai bước:

- **fit():** tìm tất cả các giá trị khác nhau xuất hiện trong cột `savings_goal_met`.

- **transform():** chuyển từng giá trị thành một số nguyên tương ứng.

```
# Mục tiêu 1 cho Tầng Phân loại (0 = Không đạt, 1 = Đạt mục tiêu)
le = LabelEncoder()
y_class = le.fit_transform(df['savings_goal_met'])
```

Ví dụ:

<i>Giá trị gốc</i>	<i>Sau mã hóa</i>
No	0
Yes	1

Việc chuyển đổi này giúp các mô hình Logistic Regression, Decision Tree và Random Forest có thể xử lý dữ liệu đầu ra.

4.2.2. Lưu tên các lớp

Ngay sau khi mã hóa, chương trình lưu lại tên của các lớp:

```
target_names = le.classes_ # Lưu lại tên các nhãn gốc để dùng sau này
```

Thuộc tính `classes_` chứa toàn bộ tên các lớp ban đầu trước khi mã hóa.

Ví dụ:

['No', 'Yes']

Thông tin này được sử dụng ở các bước sau để đặt tên cho các cột xác suất dự đoán khi thực hiện kỹ thuật Feature Stacking.

Nếu không lưu lại tên lớp, chương trình chỉ biết các giá trị 0 và 1 mà không biết chúng tương ứng với "Yes" hay "No".

4.2.3. Xây dựng biến mục tiêu cho bài toán hồi quy

Khác với bài toán phân loại, bài toán hồi quy cần dự đoán giá trị số liên tục. Trong chương trình, biến mục tiêu được lựa chọn là:

```
# Mục tiêu 2 cho Tầng Hồi quy (Dự đoán số tiền tiết kiệm thực tế)
y_reg = df['actual_savings']
```

Cột **actual_savings** lưu số tiền tiết kiệm thực tế của từng khách hàng. Đây là biến liên tục nên không cần thực hiện mã hóa mà có thể sử dụng trực tiếp trong mô hình RandomForestRegressor.

Như vậy sau bước này chương trình đã xây dựng đầy đủ hai biến mục tiêu:

- **y_class** dùng cho bài toán phân loại.
- **y_reg** dùng cho bài toán hồi quy.

Việc tách riêng hai biến mục tiêu giúp chương trình có thể xây dựng mô hình nhiều tầng (Multi-stage Machine Learning), trong đó tầng đầu giải quyết bài toán phân loại, còn tầng sau giải quyết bài toán hồi quy.

4.3. CHIA TẬP DỮ LIỆU VÀ CHUẨN HÓA DỮ LIỆU

Sau khi xác định được tập dữ liệu đầu vào và các biến mục tiêu, chương trình tiến hành chia dữ liệu thành tập huấn luyện và tập kiểm tra. Đây là bước quan trọng nhằm đánh giá khả năng dự đoán của mô hình trên các dữ liệu chưa từng xuất hiện trong quá trình học.

4.3.1. Chia dữ liệu Train và Test

Chương trình sử dụng hàm `train_test_split()` của Scikit-learn.

```
# Chia tập Train/Test (Tỷ lệ 80/20, giữ phân tầng lớp)
X_train, X_test, y_class_train, y_class_test, y_reg_train, y_reg_test = train_test_split(
    X_base, y_class, y_reg,
    test_size=0.2,
    random_state=42,
    stratify=y_class
)
```

Hàm `train_test_split()` đồng thời chia ba tập dữ liệu:

- `X_base`
- `y_class`
- `y_reg`

Để đảm bảo dữ liệu đầu vào luôn tương ứng với hai biến mục tiêu. Các tham số được lựa chọn gồm:

- **`test_size = 0.2`**
 - Tham số này quy định 20% dữ liệu sẽ được sử dụng để kiểm tra mô hình, còn 80% dữ liệu dùng để huấn luyện.
 - Tỷ lệ 80/20 là tỷ lệ được sử dụng rất phổ biến trong các bài toán Machine Learning vì vừa đảm bảo mô hình có đủ dữ liệu để học vừa giữ lại một lượng dữ liệu độc lập để đánh giá.
- **`random_state = 42`**
 - Việc chia dữ liệu trong Machine Learning được thực hiện ngẫu nhiên.
 - Nếu không cố định giá trị `random_state` thì mỗi lần chạy chương trình dữ liệu sẽ được chia khác nhau, dẫn đến kết quả Accuracy và các chỉ số đánh giá cũng thay đổi.

stratify = y_class

- Đây là một tham số rất quan trọng trong bài toán phân loại. Trong bộ dữ liệu, số lượng khách hàng đạt mục tiêu tiết kiệm và không đạt mục tiêu không cân bằng.
- Nếu chia dữ liệu hoàn toàn ngẫu nhiên thì có thể xảy ra trường hợp tập Test chỉ chứa rất ít mẫu của một lớp.
- Giúp giữ nguyên tỷ lệ của từng lớp trong cả tập Train và Test.

Ví dụ:

Nếu dữ liệu gốc gồm:

- 90% lớp No
- 10% lớp Yes

thì sau khi chia Train và Test vẫn giữ gần đúng tỷ lệ này. Điều này giúp mô hình được đánh giá khách quan hơn.

4.3.2. Chuẩn hóa dữ liệu

Các đặc trưng tài chính trong bộ dữ liệu có đơn vị đo rất khác nhau.

Ví dụ:

- Credit Score khoảng 300–850.
- Monthly Income có thể lên đến hàng chục nghìn USD.
- Savings Rate chỉ dao động từ 0 đến 1.

Nếu đưa trực tiếp vào mô hình thì những biến có giá trị lớn sẽ ảnh hưởng nhiều hơn trong quá trình học.

Để khắc phục điều này chương trình sử dụng **StandardScaler**.

```
# Chuẩn hóa dữ liệu gốc (Z-score Scaling)
scaler = StandardScaler()
```

Đây là phương pháp chuẩn hóa dữ liệu theo phân phối chuẩn (Z-score).

4.3.2.1. Chuẩn hóa tập Train

```
X_train_scaled = scaler.fit_transform(X_train)
```

Lệnh trên gồm hai bước:

fit(): Tính giá trị trung bình (Mean) và độ lệch chuẩn (Standard Deviation) của từng đặc trưng trong tập Train.

transform(): Áp dụng công thức để chuẩn hóa toàn bộ dữ liệu.

$$Z = \frac{X - \mu}{\sigma}$$

Sau chuẩn hóa:

- Mean ≈ 0
- Standard Deviation ≈ 1

Điều này giúp các thuật toán học nhanh hơn và ổn định hơn.

4.3.2.2. Chuẩn hóa tập Test

```
X_test_scaled = scaler.transform(X_test)
```

Khác với tập Train, tập Test **không sử dụng fit()**. Nếu thực hiện fit() trên tập Test thì mô hình sẽ biết trước thông tin của dữ liệu kiểm tra, gây ra hiện tượng **Data Leakage** (rò rỉ dữ liệu).

Do đó chương trình chỉ sử dụng: transform() để chuẩn hóa tập Test bằng chính Mean và Standard Deviation đã tính từ tập Train.

Đây là quy trình chuẩn trong mọi bài toán Machine Learning.

```
# Chuẩn hóa dữ liệu gốc (Z-score Scaling)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

4.4. XÂY DỰNG VÀ HUẤN LUYỆN CÁC MÔ HÌNH HỌC MÁY

Sau khi hoàn thành bước chuẩn hóa dữ liệu, chương trình tiến hành xây dựng các mô hình học máy nhằm dự đoán khả năng đạt mục tiêu tiết kiệm của khách hàng.

4.4.1. Khởi tạo các mô hình

Trong chương trình, ba mô hình được khai báo bằng Dictionary:

```
# Khai báo danh sách các mô hình học máy phân loại
models = {
    "Logistic Regression": LogisticRegression(max_iter=2000, class_weight='balanced', random_state=42),
    "Decision Tree": DecisionTreeClassifier(max_depth=10, class_weight='balanced', random_state=42),
    "Balanced Random Forest": RandomForestClassifier(n_estimators=200, class_weight='balanced', max_depth=10, random_state=42)
}
```

Việc lưu các mô hình trong Dictionary giúp chương trình có thể sử dụng cùng một vòng lặp để huấn luyện và đánh giá tất cả các mô hình mà không cần viết lại nhiều đoạn mã giống nhau.

4.4.1.1. Logistic Regression

Mô hình Logistic Regression được khởi tạo:

```
"Logistic Regression": LogisticRegression(max_iter=2000, class_weight='balanced', random_state=42),
```

Trong đó:

- **max_iter = 2000**: tăng số vòng lặp tối đa để mô hình có đủ thời gian hội tụ.

- **class_weight = balanced**: tự động điều chỉnh trọng số giữa các lớp nhằm giảm ảnh hưởng của dữ liệu mất cân bằng.
- **random_state = 42**: giúp kết quả huấn luyện có thể tái lập.

4.4.1.2. *Decision Tree*

```
"Decision Tree": DecisionTreeClassifier(max_depth=10, class_weight='balanced', random_state=42),
```

Tham số **max_depth = 10** giới hạn độ sâu của cây nhằm tránh hiện tượng Overfitting. Đồng thời `class_weight='balanced'` giúp cây quyết định chú ý nhiều hơn đến lớp thiểu số.

4.4.1.3. *Balanced Random Forest*

```
"Balanced Random Forest": RandomForestClassifier(n_estimators=200, class_weight='balanced', max_depth=10, random_state=42)
```

Mô hình sử dụng **200 cây quyết định**, sau đó tổng hợp kết quả dự đoán thông qua cơ chế bỏ phiếu đa số. Nhờ kết hợp nhiều cây khác nhau, Random Forest thường có khả năng tổng quát hóa tốt hơn Decision Tree đơn lẻ và giảm đáng kể hiện tượng quá khớp.

4.4.2. *Huấn luyện và đánh giá*

Chương trình sử dụng một vòng lặp:

```
# Duyệt qua từng mô hình để huấn luyện, đo thời gian và in báo cáo chuẩn
for name, model in models.items():
    model_start = time.time() # Bắt đầu bấm giờ cho riêng mô hình này

    # Huấn luyện mô hình phân loại dựa trên tập Train
    model.fit(X_train_scaled, y_class_train)
    y_pred = model.predict(X_test_scaled)

    model_end = time.time() # Dừng bấm giờ mô hình

    # Xuất báo cáo y hệt định dạng mong muốn
    print(f"▶ {name} Report:")
    print(classification_report(y_class_test, y_pred, digits=2))
    print(f"Thời gian thực thi: {model_end - model_start:.2f} giây")
    print("-" * 70)

    # Trích xuất xác suất dự đoán (%) để làm đầu vào cho Tầng 2
    train_probs_list.append(model.predict_proba(X_train_scaled))
    test_probs_list.append(model.predict_proba(X_test_scaled))
```

for name, model in models.items(): để lần lượt huấn luyện từng mô hình.

Trong mỗi lần lặp:

- model.fit() dùng để huấn luyện mô hình trên tập Train đã chuẩn hóa.
- model.predict() tạo dự đoán trên tập Test.
- classification_report() tính Accuracy, Precision, Recall và F1-score.
- predict_proba() sinh ra xác suất dự đoán của từng lớp. Đây là dữ liệu rất quan trọng vì sẽ được sử dụng ở bước **Feature Stacking** trong tầng hồi quy của hệ thống.

Cách tổ chức này giúp chương trình ngắn gọn, dễ mở rộng và đảm bảo tất cả các mô hình đều được đánh giá trên cùng một tập dữ liệu với cùng quy trình, tạo điều kiện thuận lợi cho việc so sánh hiệu quả giữa các thuật toán.

CHƯƠNG 5: ĐÁNH GIÁ VÀ NHẬN XÉT MÔ HÌNH HỌC MÁY

5.1. ĐÁNH GIÁ MÔ HÌNH LOGISTIC REGRESSION

Sau khi huấn luyện mô hình Logistic Regression trên tập dữ liệu huấn luyện và kiểm tra trên tập dữ liệu kiểm tra chúng ta thu được các chỉ số đánh giá như sau

5.1.1. Kết quả mô hình Logistic Regression

Chỉ số	Giá trị
Accuracy	88%
Precision lớp 0	0.99
Recall lớp 0	0.88
F1-score lớp 0	0.93
Precision lớp 1	0.42
Recall lớp 1	0.91
F1-score lớp 1	0.58
Macro Average F1	0.75
Weighted Average F1	0.90
Thời gian huấn luyện	0.01 giây

5.1.2. Nhận xét

- Mô hình Logistic Regression đạt **độ chính xác tổng thể (Accuracy)** là **88%**, cho thấy mô hình có khả năng phân loại khá tốt đối với bài toán dự đoán khả năng đạt mục tiêu tiết kiệm.

- Đối với lớp 0 (không đạt mục tiêu tiết kiệm), mô hình đạt Precision rất cao (**0.99**) và Recall đạt **0.88**, chứng tỏ phần lớn các dự đoán thuộc lớp này đều chính xác.
 - Đối với lớp 1 (đạt mục tiêu tiết kiệm), Recall đạt **0.91**, nghĩa là mô hình phát hiện được phần lớn khách hàng thực sự đạt mục tiêu. Tuy nhiên Precision chỉ đạt **0.42**, cho thấy vẫn còn khá nhiều trường hợp dự đoán nhầm sang lớp 1.
- ⇒ Điều này xuất phát từ việc dữ liệu mất cân bằng (545 mẫu lớp 0 và chỉ 55 mẫu lớp 1), khiến mô hình ưu tiên phát hiện đầy đủ lớp thiểu số nhưng đánh đổi bằng số lượng dự đoán sai cao hơn.

5.2. ĐÁNH GIÁ MÔ HÌNH DECISION TREE

5.2.1. Kết quả mô hình Decision Tree

Chỉ số	Giá trị
Accuracy	89%
Precision lớp 0	0.95
Recall lớp 0	0.92
F1-score lớp 0	0.94
Precision lớp 1	0.41
Recall lớp 1	0.55
F1-score lớp 1	0.47
Macro Average F1	0.70
Weighted Average F1	0.89
Thời gian huấn luyện	0.01 giây

5.2.2. Nhận xét

- Mô hình Decision Tree đạt Accuracy **89%**, cao hơn Logistic Regression khoảng **1%**.
 - Mô hình dự đoán rất tốt lớp 0 với Precision **95%** và Recall **92%**.
 - Tuy nhiên, hiệu quả trên lớp 1 chưa cao. Recall chỉ đạt **55%**, nghĩa là mô hình chỉ nhận diện được khoảng một nửa số khách hàng thực sự đạt mục tiêu tiết kiệm.
 - F1-score của lớp 1 chỉ đạt **0.47**, cho thấy mô hình vẫn còn gặp khó khăn khi xử lý dữ liệu mất cân bằng.
- ⇒ Ưu điểm lớn của Decision Tree là tốc độ huấn luyện rất nhanh và mô hình dễ giải thích thông qua cấu trúc cây quyết định.

5.3. ĐÁNH GIÁ MÔ HÌNH BALANCED RANDOM FOREST

5.3.1. Kết quả mô hình *Balanced Random Forest*

Chỉ số	Giá trị
Accuracy	93%
Precision lớp 0	0.98
Recall lớp 0	0.94
F1-score lớp 0	0.96
Precision lớp 1	0.56
Recall lớp 1	0.80
F1-score lớp 1	0.66
Macro Average F1	0.81
Weighted Average F1	0.93
Thời gian huấn luyện	0.45 giây

5.3.2. Nhận xét

- Balanced Random Forest là mô hình cho kết quả tốt nhất trong ba mô hình phân loại.
- Độ chính xác đạt **93%**, cao hơn:
 - Logistic Regression: **5%**
 - Decision Tree: **4%**

- Đối với lớp 1, Recall đạt **80%**, cao hơn đáng kể so với Decision Tree (55%).
- F1-score lớp 1 đạt **0.66**, cho thấy mô hình xử lý tốt hơn đối với dữ liệu mất cân bằng.
- Mặc dù thời gian huấn luyện (**0.45 giây**) dài hơn hai mô hình còn lại, nhưng vẫn rất nhỏ và hoàn toàn chấp nhận được.
- Balanced Random Forest cho khả năng tổng quát hóa tốt nhờ kết hợp nhiều cây quyết định, từ đó giảm hiện tượng quá khớp và tăng độ ổn định của kết quả dự đoán.

5.4. ĐÁNH GIÁ MÔ HÌNH HỒI QUY

Sau khi hoàn thành giai đoạn phân loại, đề tài tiếp tục sử dụng kỹ thuật **Feature Stacking** để kết hợp các xác suất dự đoán từ ba mô hình phân loại với dữ liệu gốc, sau đó huấn luyện mô hình RandomForestRegressor nhằm dự đoán số tiền tiết kiệm thực tế.

```
meta_start = time.time() # Bấm giờ cho tầng hồi quy

# Khởi tạo mô hình Random Forest dạng Hồi quy gồm 300 cây
regressor = RandomForestRegressor(n_estimators=300, max_depth=15, random_state=42)
regressor.fit(X_train_hybrid, y_reg_train) # Huấn luyện dựa trên tập dữ liệu lai ghép (gốc + xác suất)
reg_preds = regressor.predict(X_test_hybrid) # Dự đoán trên tập kiểm tra

# Đánh giá Meta-Model bằng các chỉ số cốt lõi
r2 = r2_score(y_reg_test, reg_preds)
mae = mean_absolute_error(y_reg_test, reg_preds)
rmse = np.sqrt(mean_squared_error(y_reg_test, reg_preds)) # Sai số bình phương trung bình (RMSE) giúp bắt lỗi lệch lớn

meta_end = time.time()

print(f"► Meta-Model RandomForest Regressor:")
print(f" -> Hệ số xác định (R² Score) : {r2*100:.2f}%")
print(f" -> Sai số tiền dự đoán trung bình (MAE) : ${mae:.2f}")
print(f" -> Sai số căn lẽ bình phương (RMSE) : ${rmse:.2f}")
print(f"Thời gian thực thi: {meta_end - meta_start:.2f} giây")
print("-" * 70)
```

Chỉ số	Giá trị
R ² Score	96.07%
MAE	73.45 USD
RMSE	204.41 USD
Thời gian huấn luyện	2.98 giây

Kết quả cho thấy mô hình hồi quy đạt **R² Score = 96.07%**, nghĩa là mô hình có thể giải thích khoảng **96% sự biến thiên của số tiền tiết kiệm thực tế**. Đây là một mức rất cao, chứng tỏ mô hình phù hợp với dữ liệu.

Sai số tuyệt đối trung bình (**MAE**) chỉ khoảng **73.45 USD**, tức trung bình mỗi dự đoán chỉ sai lệch khoảng 73 USD so với giá trị thực tế.

Sai số RMSE đạt **204.41 USD**. Do RMSE phạt mạnh các sai số lớn nên giá trị này phản ánh mô hình vẫn tồn tại một số trường hợp dự đoán lệch đáng kể, tuy nhiên nhìn chung hiệu quả dự đoán vẫn rất tốt.

Thời gian huấn luyện khoảng **2.98 giây**, phù hợp với mô hình Random Forest có số lượng cây lớn (**300 cây**) và bộ dữ liệu được mở rộng bằng kỹ thuật Feature Stacking.

5.5. SO SÁNH CÁC MÔ HÌNH

5.5.1. So sánh các mô hình phân loại

Tiêu chí	Logistic Regression	Decision Tree	Balanced Random Forest
Accuracy	88%	89%	93%
Precision lớp 1	0.42	0.41	0.56
Recall lớp 1	0.91	0.55	0.80
F1-score lớp 1	0.58	0.47	0.66
Weighted F1-score	0.90	0.89	0.93
Thời gian huấn luyện	0.01 s	0.01 s	0.45 s

5.5.2. Nhận xét

Qua kết quả thực nghiệm có thể thấy mỗi mô hình đều có những ưu điểm riêng.

- **Logistic Regression** có tốc độ huấn luyện nhanh và khả năng phát hiện lớp đạt mục tiêu tiết kiệm rất tốt (Recall = 0.91), tuy nhiên Precision còn thấp do dữ liệu mất cân bằng.
- **Decision Tree** có cấu trúc đơn giản, dễ giải thích nhưng khả năng nhận diện lớp thiểu số còn hạn chế.

- **Balanced Random Forest** đạt kết quả tốt nhất với Accuracy 93%, F1-score cao nhất và khả năng xử lý dữ liệu mất cân bằng hiệu quả, phù hợp để triển khai trong hệ thống dự đoán.

Đối với bài toán hồi quy, mô hình **RandomForestRegressor** kết hợp kỹ thuật **Feature Stacking** đạt R^2 Score lên tới **96.07%**, cùng sai số MAE và RMSE ở mức thấp. Điều này chứng minh việc bổ sung các xác suất dự đoán từ tầng phân loại đã giúp cải thiện đáng kể chất lượng dự đoán số tiền tiết kiệm thực tế.

CHƯƠNG 6: TỔNG KẾT

Qua quá trình thực hiện đề tài, nhóm chúng em đã hoàn thành các mục tiêu đề ra: từ việc xây dựng quy trình ETL làm sạch dữ liệu nhiễu, phân tích trực quan hóa (EDA) gắn liền với thực tiễn tài chính, đến việc thử nghiệm mô hình học máy 2 tầng (Feature Stacking) và lưu trữ dữ liệu vào MySQL. Mặc dù đạt được những kết quả nhất định, báo cáo của nhóm vẫn còn những hạn chế như dữ liệu mới dừng lại ở mức giả lập, các thuật toán còn cơ bản và chưa xây dựng được giao diện trực quan cho người dùng. Nhóm chúng em xin chân thành cảm ơn Thầy đã tận tình hướng dẫn và đồng hành cùng nhóm trong suốt môn học này!

TÀI LIỆU THAM KHẢO

1. **McKinney, W. (2012).** *Python for Data Analysis: Data Wrangling with Pandas, NumPy, and IPython*. O'Reilly Media.
2. **Scikit-learn Developers (2011).** *Scikit-learn: Machine Learning in Python*.
3. **Pandas Development Team (2020).** *pandas: powerful Python data analysis toolkit*.
4. **NumPy Developers (2020).** *NumPy User Guide*.
5. **Github Nhóm 6**